Đánh giá giá trị bổ sung và chẩn đoán hệ thống GRACE

Đề cương nghiên cứu tổng hợp và hướng dẫn triển khai thực nghiệm

Ngày 28 tháng 10 năm 2026

Tóm tắt

Định hướng nghiên cứu là đo giá trị mà nhánh graph và demonstration của LLM bổ sung cho một bộ phân loại nhẹ đã khóa, đồng thời dùng các phép tái lập có chẩn đoán để giải thích kết quả. Trục chính tận dụng H3-B và artifact hiện có; các probe graph, nội dung demo, nhãn và điểm số liên tục được bổ sung theo từng tầng ngân sách. Không coi mô hình phối hợp đã thành công chỉ vì classifier độc lập vượt GRACE, và không coi kết quả chưa có lợi ích là bằng chứng LLM hoàn toàn không mang thông tin bổ sung.

Trên các cohort lịch sử đã tiếp xúc trong phát triển, classifier hoặc fusion được chọn có MCC 0,2961 trên Devign và 0,3705 trên ReVeal, trong khi các nhánh phối hợp chưa xác nhận lợi ích tăng thêm. Expert vẫn sửa được một số lỗi classifier theo số đếm đối soát, nên cần phân biệt tính bổ sung có sẵn, giá trị khai thác sau truy vấn và khả năng chọn query hữu ích trước truy vấn. Bài nghiên cứu đề xuất cặp meta LR có và không có feature expert, cùng gate/ngưỡng/folds; routing ở cùng ngân sách sample; audit nội dung prompt; và một cohort xác nhận ngoài có thiết kế lấy mẫu rõ ràng.

Các đối chứng mới gồm graph sai ghép hoặc đổi quan hệ, demo ngẫu nhiên có kiểm soát nhãn/độ dài, bỏ nhãn, chỉ nhãn, đảo nhãn, repeat và đổi thứ tự cặp. Score từ logprob được dùng khi đủ candidate thật ở đúng prefix; thiếu thông tin không được bù bằng xác suất giả. Tự host là một phương án hạ tầng có điều kiện, không phải yêu cầu bắt buộc hay bảo đảm tất định.

Tài liệu có quy trình thực nghiệm theo từng bước, bản đồ cập nhật tại các hàm nguồn đã kiểm, gói mã/cấu hình dùng được cho phân tích offline và các kiểm thử kỹ thuật. Một mục riêng thống kê đúng 26 khả năng suy luận không được chấp nhận hoặc cần giới hạn lại, với phân tích chi tiết. Gói lõi được dự toán khoảng ba đến bốn tuần khi artifact và endpoint sẵn sàng; thêm hạ tầng/scoring và chẩn đoán mở rộng có thể cần sáu đến tám tuần. Mục tiêu công bố là nghiên cứu thực nghiệm có câu hỏi rõ, đối chứng đúng và bằng chứng mới; không có bảo đảm acceptance hoặc Q1 chỉ bằng một lịch trình.

Từ khóa Phát hiện lỗ hổng phần mềm; GRACE; tái lập có chẩn đoán; giá trị bổ sung; learning to defer; đánh giá ghép cặp.

Nội dung

Mục 1 đến 5 xác lập câu hỏi và phép so chính. Mục 6 đến 12 hướng dẫn can thiệp, đo lường và thực nghiệm. Mục 15 phân tích 26 khả năng suy luận chưa được chấp nhận. Mục 19 và các phụ lục hướng dẫn cập nhật phần mềm.

1 Bối cảnh và quyết định nghiên cứu 3
2 Kết quả thăm dò và vấn đề còn bỏ ngỏ 4
3 Nghiên cứu liên quan và khoảng trống cần xác lập 7
4 Phát biểu bài toán và câu hỏi nghiên cứu 8
5 Thiết kế hệ thống nghiên cứu tối thiểu 10
6 Chẩn đoán graph và demonstration bằng các phép can thiệp có kiểm soát 11
7 Điểm số liên tục và lựa chọn backend 14
8 Đo tính bổ sung và chi phí một cách đúng nghĩa 16
9 Dữ liệu và thiết kế xác nhận 17
10 Ma trận thực nghiệm tối thiểu 19
11 Quy trình thực nghiệm và cập nhật phần mềm theo từng bước 21
12 Thống kê và tiêu chí kết luận 24
13 Kế hoạch thời gian và nguồn lực 25
14 Điều kiện dừng và cách xử lý kết quả âm 26
15 Các khả năng suy luận chưa được chấp nhận 26
16 Định vị bài hội thảo và lộ trình mở rộng tạp chí 33
17 Rủi ro đối với độ tin cậy của nghiên cứu 35
18 Sản phẩm cần bàn giao 36
19 Bản đồ cập nhật mã nguồn hiện có 36
Phụ lục A Gói mã và cấu hình đi kèm 39
Phụ lục B Các lệnh thực hiện và đầu vào minh họa 40
Phụ lục C Mẫu bảng hình và nghiệm thu 40
Tài liệu nguồn thực nghiệm 41
Tài liệu tham khảo 41

1 Bối cảnh và quyết định nghiên cứu

1.1 Bài toán và lý do cần đổi trọng tâm

Trong phát hiện lỗ hổng ở mức hàm, đầu vào là mã nguồn C/C++ và đầu ra là nhãn có hoặc không có lỗ hổng theo nhãn benchmark. GRACE bổ sung biểu diễn graph và demonstration truy hồi vào prompt của LLM [T1]. Về nguyên tắc, graph có thể làm rõ quan hệ chương trình, còn demonstration có thể đưa ví dụ gần với target. Nhưng một thành phần hợp lý về thiết kế chưa chắc cung cấp thông tin hữu ích trong prompt thực tế, và thêm thông tin chưa chắc làm dự đoán cuối tốt hơn.

Sau đối soát, nhóm đã chuyển từ so sánh một vài con số tổng hợp sang lưu run, nhãn từng mẫu, raw response, hash và các đối chứng. Đây là nền tốt để hỏi một câu hỏi nhỏ hơn nhưng sâu hơn: **so với một bộ phân loại nhẹ đã được chọn công bằng, phần nào của chi phí LLM tạo ra lợi ích đo được?** Câu hỏi này phù hợp hơn việc tiếp tục thêm loss, thêm view hoặc thêm prompt để tìm một cấu hình có F1 cao.

Trong tài liệu này, “expert” là tên vai trò của nhánh GRACE/LLM; từ này không hàm ý nhánh đó chính xác hơn classifier. “Gate” được dùng theo tên mã hiện có, nhưng khi không thực hiện routing thì nó đơn giản là một bộ phân loại độc lập. “Demo” hay “demonstration” là ví dụ mã nguồn có nhãn lấy từ kho train; nhãn target chỉ được ghép ở bước đánh giá.

1.2 Các tài liệu và nguồn thực nghiệm

Phản hồi chính thức [N1] cung cấp các nguyên tắc cần giữ: đúng nguồn gốc run; đối chứng thành phần; giữ ca lỗi; đánh giá ghép cặp; không dùng test để chọn phương án. Báo cáo chung [N2] cho biết tiến độ đối soát và kết quả development. Báo cáo cải tiến cập nhật ngày 24/10 [N3] là nguồn chính cho việc lựa chọn giữa H1, H2, H3-A và H3-B.

Các bảng “kết quả hiện có” dưới đây trích số liệu trong [N2–N3]. Bản tổng hợp chuyên môn [N4] bổ sung các giả thuyết và phép chẩn đoán cần kiểm tra; các giả thuyết này không được coi là kết quả thực nghiệm đã có. Những số suy ra từ số đếm được ghi là tái tính. Trước khi đưa vào bài nộp, cần tạo lại chúng từ prediction canonical và cùng một evaluator. Một đường dẫn ổ đĩa D trong báo cáo là chỉ dẫn tới artifact của nhóm, không tự tạo thành gói tái lập mà người phản biện truy cập được.

1.3 So sánh bốn hướng theo công việc còn phải làm

Hướng	Tín hiệu đang có	Phần việc lớn còn thiếu	Quyết định cho bài đầu
H1-V2 học projection	ReVeal paired ΔMCC 0,0019; CI chứa 0	Fitter, repeat control, Devign V2, residual-CE và ablation loss	Tạm dừng mở rộng
H2 giữ evidence và guard	Có 5.006 prediction full test	Prefix/slice control cùng điều kiện, kiểm đúng graph và guard, đối chiếu tính mới	Giữ làm mở rộng có điều kiện
H3-A cascade hiệu chuẩn	Bỏ qua 6/2.732 Devign; 0/2.274 ReVeal	Đánh đổi risk–coverage, kiểm chứng ngoài, so selective baseline	Dùng như kết quả đối chứng
H3-B classifier và phối hợp	Classifier có tín hiệu rõ; đủ nhiều nhánh và artifact	Đối chứng cùng gate, kiểm expert shift, holdout mới, chi phí và phân tích lỗi	Chọn làm nền nghiên cứu

H1-V1 còn có ΔMCC âm trên Devign trong phân tích ghép cặp được báo cáo. H1-V2 tăng một chỉ số retrieval gián tiếp nhưng chưa xác nhận downstream. H3-A đang tiết kiệm quá ít truy vấn để làm đóng góp chính về hiệu quả. H2 có câu hỏi phương pháp hấp dẫn, nhưng rủi ro nằm ở xác minh ngữ nghĩa guard và độ mới, không chỉ ở việc chạy thêm một baseline.

H3-B được chọn vì phần còn thiếu chủ yếu là **thiết kế phép so sánh và khai thác bằng chứng đã có**. Không chọn H3-B vì nó đã chứng minh một ensemble mới thành công. Ngược lại, việc winner không gọi LLM chính là một kết quả cần giải thích.

1.4 Phạm vi tái lập và sai khác cần ghi

Bảng replication phải đối chiếu bài gốc, mã tác giả và từng bản tái cài đặt: encoder/tokenizer và pooling; whitening fit trên đâu; thứ tự common pool/top-k/rerank; lexical/AST distance; Joern/version/schema; graph source và renderer; vị trí các block trong prompt; byte/token; số demo; output/scoring protocol. Mỗi dòng ghi đúng phần đã xác nhận và phần còn chưa xác định. Không ép một thứ tự block thành chân lý cho mọi implementation; một thay đổi thứ tự cần là control riêng hoặc được khai báo như sai khác toàn pipeline.

Hướng được chọn kết hợp nghiên cứu giá trị bổ sung với diagnostic replication, không theo đuổi đồng thời bốn cải tiến như bốn bài riêng. Kết quả H1, H2 và H3-A được dùng ở vai trò mốc lịch sử, can thiệp giải thích hoặc nhánh mở rộng khi bằng chứng chỉ ra nhu cầu. “Dừng mở rộng” không có nghĩa xóa công sức hay kết luận hướng đó bất khả thi.

2 Kết quả thăm dò và vấn đề còn bỏ ngỏ

2.1 Tín hiệu classifier và giới hạn của kết quả hiện tại

Dataset và nhánh	MCC	Balanced accuracy	F1	Truy vấn expert theo thuật toán
Devign full GRACE	0,0668	52,57%	29,80%	100%
Devign gate_tuned đã khóa	0,2961	64,23%	57,45%	0%
Devign stacking LR	0,3013	64,62%	58,60%	100%
ReVeal full GRACE	0,0971	54,26%	17,41%	100%
ReVeal gate_tuned đã khóa	0,3705	73,13%	43,62%	0%
ReVeal stacking LR	0,3437	67,83%	41,25%	100%

Nguồn: [N3], mục 4.6 và 6.2.2, session DeepSeek seed 42 ngày 24/10/2026. Trên Devign, stacking LR trừ gate_tuned có ΔMCC 0,0053, CI 95% [−0,0186; 0,0276]. Trên ReVeal, ΔMCC là −0,0268, CI [−0,0684; 0,0139]. Các CI này theo query trong báo cáo, chưa xử lý đầy đủ phụ thuộc project/commit.

Control gate_0_5 có MCC 0,3025 trên Devign, cao hơn gate_tuned theo điểm. Vẫn giữ winner đã khóa là gate_tuned; không chọn lại bằng test. Tương tự, “0% query expert” chỉ mô tả suy luận của winner. Việc học/chọn phương án H3-B trước đó vẫn sử dụng expert development và có chi phí.

2.2 Expert yếu hơn về trung bình vẫn có thể sửa lỗi classifier

Gọi R là số mẫu classifier sai nhưng expert đúng, H là số mẫu classifier đúng nhưng expert sai. Đối với việc thay classifier bằng expert trên toàn cohort, R là số sửa đúng và H là số làm sai. Bảng đối soát [N3, mục 6.2] đang ghi số sửa đúng/làm sai khi đi theo chiều ngược lại, từ full GRACE sang gate; cần đổi chiều đúng khi sử dụng.

Đại lượng tái tính từ số đếm báo cáo	Devign	ReVeal
Classifier đúng	1.784/2.732	1.925/2.274
Expert sửa lỗi classifier R	303	218
Expert làm sai quyết định đúng của classifier H	580	201
Accuracy nếu biết nhãn thật để chọn bên đúng	76,39%	94,24%

Dòng cuối là một **trần chẩn đoán về accuracy**, dùng nhãn thật để chọn predictor đúng khi một trong hai đúng. Nó không phải phương pháp triển khai và không phải trần tối ưu MCC. Các số này cần được xác nhận từ từng ID trước công bố. Chúng cho thấy câu hỏi "expert có bao giờ giúp không" khác với câu hỏi "router có biết khi nào nên dùng expert không".

ReVeal còn cho thấy một sự khác biệt về mục tiêu: full GRACE có accuracy 85,40%, cao hơn gate_tuned 84,65%, nhưng MCC thấp hơn nhiều. Vì vậy, học "lợi ích" bằng lỗi 0/1 có thể ưu tiên một sự thay đổi không phù hợp với MCC hoặc cân bằng lớp. Hiện tượng này đáng kiểm tra; chưa đủ kết luận đó là nguyên nhân khiến benefit-router chưa thắng.

2.3 Những điểm phải xử lý trước khi suy luận cơ chế

Thứ nhất, router/selector hiện dùng gate với ngưỡng chọn trên train, trong khi headline gate_tuned dùng ngưỡng khác chọn trên development. Đây là so sánh các policy hoàn chỉnh, chưa cô lập tác dụng của expert. Lượt mới cần một classifier, một ngưỡng chung và một bộ đặc trưng chung ở các phép so tương ứng.

Thứ hai, stacker hiện có 38 feature và một lần học trên development, còn gate không có tầng học tương ứng. Nếu stacker thắng gate thì chưa biết lợi ích đến từ expert, từ 34 feature có trước expert hay từ việc học thêm. Cần cặp meta-model có và không có bốn feature expert.

Thứ ba, expert development và test trong session được báo cáo qua hai provider/alias, với trường thinking chưa đầy đủ. Giữ cờ expert_shift và benchmark_comparable=false cho session lịch sử. Điều này không xóa giá trị số liệu, nhưng hạn chế kết luận về một expert ổn định. Lượt xác nhận phải dùng cùng cấu hình phục vụ cho development và test.

Thứ tư, H1 có nhiều demo bị cắt rỗng và có prompt giống nhau nhưng prediction khác nhau. Không chuyển nguyên tỷ lệ này sang H3-B; phải audit chính prompt H3-B. Demo ID khác không có nghĩa model nhận được code khác. "Temperature 0, seed 42" cũng không đủ để mặc định response lặp lại hoàn toàn.

Thứ năm, test đã được xem. Phần test không trùng exact code với train/development dự kiến còn 2.727 Devign và 2.211 ReVeal. Lọc các mẫu trùng giúp phân tích độ nhạy, nhưng không biến test cũ thành holdout độc lập.

2.4 Những nhận xét trong báo cáo chung cần giữ ở mức giả thuyết

Tên gọi "over-sensitization" mô tả việc TP/FP cùng tăng; nó chưa chứng minh nguyên nhân là cặp demo 1:1 hoặc backbone thiếu năng lực suy luận. Đổi sang model khác là một can thiệp cần đo, không phải lời giải đã biết. Tương tự, CI của B3 so B2 về MCC chứa 0 chưa xác nhận tác dụng dù F1 tăng.

Bộ chấm retrieval có 50 target mỗi dataset, agreement ban đầu 24/50 và κ khoảng 0,32. Ưu thế sau hòa giải trên ReVeal là tín hiệu thăm dò; "Neither" trên Devign chưa đủ chứng minh kho train không có mẫu hữu ích. Nguyên nhân có thể thuộc retriever, rubric, thiếu ngữ cảnh hoặc giới hạn người chấm. Bài mới không lấy các diễn giải mạnh này làm tiền đề đã được xác nhận.

2.5 Không gộp các audit khác phạm vi

108 hàm Devign và 52 ReVeal trong một audit trùng cũ không tự bằng số test overlap với tập học/chọn H3-B. Audit H3-B ghi 5 Devign và 63 ReVeal test trùng code với train/development. Devign official validation gốc là 2.732, còn 2.724 là development đủ điều kiện sau lọc trong snapshot H3-B. Số 6 ca cùng prompt nhưng prediction khác thuộc một kiểm tra trước; bảng cập nhật ReVeal V2 ghi 77 trong 2.179 prompt giống. Mỗi số phải mang cohort, snapshot và mẫu số, không chọn một số thay thế mọi phạm vi.

MCC thấp và F1 dưới đối chứng hằng là tín hiệu để kiểm thiết kế, không phải chứng minh predictor ngẫu nhiên. Không gán MCC của winner classifier/fusion cho hồi quy logistic thuần khi chưa đọc manifest base winner. Các phân tích trong mục 15 giải thích chi tiết những khả năng suy luận này.

3 Nghiên cứu liên quan và khoảng trống cần xác lập

3.1 Những ý tưởng đã có tiền lệ

GRACE đã kết hợp graph và ICL [T1]. DLAP kết hợp mô hình học sâu với prompting [T2]; CodeRAG sử dụng truy hồi đa phương thức và quyết định phân cấp [T3]. Learning to defer và các phương pháp post-hoc đã nghiên cứu việc chuyển quyết định sang expert có chi phí [T4–T5]. Do đó, classifier trước LLM, nhiều view, hiệu chuẩn, routing và stacking không tự tạo thành tính mới.

Nghiên cứu ensemble LLM cho phát hiện lỗ hổng [T6] và RALE [T7] cũng cho thấy tính bổ sung giữa model hoặc biểu diễn đã được khai thác. Đề tài này không tuyên bố lần đầu phát hiện các predictor có lỗi khác nhau.

Các nghiên cứu gần đây đã đi xa hơn việc so một bảng F1: PrimeVul nhấn mạnh dữ liệu và chia theo thời gian [T8]; RevisitVD khảo sát đánh giá, tính khái quát và truncation [T9]; nghiên cứu code metrics cho thấy cần baseline đơn giản đủ mạnh [T10]. RAG4SVD tái lập sáu hệ RAG, gồm GRACE, trong một bối cảnh đánh giá chung [T11]. Nghiên cứu về protocol trên các cặp vulnerable/fixed đã xem xét metric, cách rút verdict và output budget khi giữ output cố định [T12].

3.2 Đối chiếu với các công trình gần nhất

Nhóm công trình	Phần đã có	Phép so đề tài cần bổ sung để có giá trị riêng
GRACE và các hướng RAG	Graph, truy hồi demo, prompting	Giá trị expert vượt đúng classifier nền, dưới contract prompt đã kiểm
DLAP và CodeRAG	Kết hợp DL–LLM, quyết định phân cấp	Tách lợi ích đổi ngưỡng, học meta và dùng đầu ra expert
Learning to defer	Học khi nào chuyển quyết định	Phân rã lỗi theo lớp và ngân sách trong bài toán lỗ hổng lệch lớp
Ensemble và RALE	Phối hợp model/biểu diễn	Phân biệt bổ sung tồn tại, bổ sung khai thác sau truy vấn và chọn trước truy vấn
PrimeVul và RevisitVD	Benchmark, leakage, độ bền	Xác nhận phép so marginal utility trên cohort ngoài đã khóa
RAG4SVD và protocol audit	Tái lập RAG, component và metric	Không lặp lại audit chung; tập trung vào lợi ích bổ sung so baseline nhẹ đủ mạnh

Trước khi chốt phần Introduction của bài nộp, lập bảng đối chiếu toàn văn cho ít nhất [T3], [T6], [T10–T12]: họ có classifier độc lập đủ mạnh không; có dùng cùng ngưỡng ở các arm không; có no-expert meta

control không; có đánh giá trước/sau truy vấn và cùng ngân sách không; có holdout khác project/thời gian không. Mỗi ô phải có trang/mục và ghi "chưa xác định" nếu chưa đối chiếu được. Không suy rằng một chi tiết không nằm trong abstract thì bài chưa làm.

Trong nhóm còn có bản Selective GRACE của nhóm công bố Selective GRACE được [N3] dẫn. Cần đối chiếu và trích dẫn bản đó như tiền lệ gần, làm rõ phần dữ liệu hoặc code dùng chung. Nếu đây là công trình của cùng nhóm nghiên cứu, cần xác định đóng góp bổ sung ngay từ đầu, tránh đổi tên một cascade đã có thành phương pháp mới.

3.3 Khoảng trống nghiên cứu dự kiến

Khoảng trống được đề xuất ở mức hẹp: **trong một hệ phát hiện lỗ hổng có classifier nhẹ và nhánh graph–ICL–LLM, chưa rõ phần lợi ích expert còn lại sau khi kiểm soát cùng classifier/ngưỡng, cùng dữ liệu học meta và cùng cấu hình phục vụ; chưa rõ phần lợi ích ấy có thể nhận diện trước truy vấn và còn tồn tại khi chuyển dữ liệu hay không**.

Đây là phạm vi cần xác lập bằng đối chiếu tài liệu và thực nghiệm, không phải tuyên bố tuyệt đối rằng chưa ai nghiên cứu vấn đề này. Nếu các công trình gần nhất đã trả lời đầy đủ, hướng phù hợp là một nghiên cứu tái lập mở rộng có câu hỏi mới về chuyển miền hoặc chi phí; không giữ tuyên bố "mới đầu tiên" bằng cách đổi thuật ngữ.

3.4 ICL và kiểm toán nhãn như nguồn đặt giả thuyết

Các công trình ICL [T20–T21] là căn cứ thiết kế flip/no-label/label-space controls, không chứng minh cơ chế sao chép ở bài toán lỗ hổng. Các nghiên cứu về chất lượng dataset và đánh giá pairwise [T8–T12] khuyến khích kiểm nhãn và patch provenance. Do đó, một kết quả liên hệ retrieval relevance với correctness phải được phân biệt với can thiệp nhân quả; khảo sát FP/FN phải công bố khung lấy mẫu trước khi suy về noise rate.

Khác biệt chính cần bảo vệ vẫn là marginal utility so cùng classifier và ngưỡng, pre/post control cùng lớp LR và khả năng router khai thác dưới chuyển dữ liệu. Không tuyên bố mới cho riêng tập probe hay tính toán MCC/AUROC.

4 Phát biểu bài toán và câu hỏi nghiên cứu

4.1 Ký hiệu và đơn vị phân tích

Mỗi mẫu có mã nguồn x, nhãn benchmark y thuộc {0,1}, ID canonical và code hash. Classifier đã khóa sinh score p và nhãn h theo ngưỡng τ. Expert sinh nhãn g hoặc trạng thái invalid. Router chỉ nhìn các đặc trưng có trước expert để quyết định có gọi expert không. Mô hình sau truy vấn được phép nhìn g nhưng phải trả chi phí expert cho mọi mẫu mà nó sử dụng.

Đơn vị ghép cặp là cùng target ID/code. Đơn vị độc lập thống kê có thể là project, commit hoặc họ clone; không mặc định mỗi hàm độc lập. Mọi kết luận ở mức hàm phải giới hạn theo dữ liệu, nhãn và ngữ cảnh hiện có, không suy thành đã phát hiện được lỗ hổng mới trong toàn repository.

4.2 Bốn câu hỏi nghiên cứu

RQ1 về tín hiệu bổ sung. Sau khi giữ nguyên classifier, dữ liệu học meta và quy trình chọn cấu hình, đầu ra expert có cải thiện được dự đoán so với cùng mô hình không dùng expert không?

RQ2 về cấu trúc lỗi. Expert sửa lỗi nào và tạo lỗi nào của classifier, riêng trên vulnerable và safe? Những khác biệt đó liên hệ với độ dài code, lượng context được giữ, loại guard và độ tương đồng demo như thế nào?

RQ3 về khai thác trước truy vấn. Ở cùng ngân sách gọi expert, benefit routing có tốt hơn chọn ngẫu nhiên và routing theo khoảng cách tới biên quyết định không? Lợi ích bổ sung có sẵn khác bao nhiêu với lợi ích router thực hiện được?

RQ4 về tính bền và chi phí. Kết quả còn giữ khi bỏ overlap, đổi seed, lặp expert và đánh giá trên cohort ngoài không? Tổng chi phí gồm học/chọn phương án, chuẩn bị và suy luận là bao nhiêu?

4.3 Phép xác nhận chính và cách tránh chọn kết quả hậu nghiệm

Phép xác nhận chính đề nghị là **meta LR có expert trừ meta LR không expert về MCC có trọng số của eligible population**, với cùng gate, cùng 34 feature nền, cùng fold và cùng quy trình chọn ngưỡng. Trên cohort ngoài, MCC có trọng số là outcome chính; MCC không trọng số là phân tích độ nhạy của cohort chẩn đoán. Đây là phép so trực tiếp nhất cho RQ1 trong lớp mô hình LR và bộ feature đã chọn, không phải kiểm định thông tin độc lập theo nghĩa lý thuyết thông tin. Lưu thêm gate-only và full GRACE để người đọc hiểu chất lượng tuyệt đối; hai mốc này không thay thế cặp chính.

Phép so routing chính đề nghị là benefit-router trừ confidence theo biên tại ngân sách 10%, dùng Δbalanced accuracy không trọng số làm outcome chính và ΔMCC làm outcome thứ cấp. Ngân sách 10% ở đây áp dụng cho cohort chẩn đoán đã lấy mẫu, không phải tỷ lệ tự nhiên của PrimeVul. Hai phép xác nhận được khai báo trước; các budget 5% và 25%, view ablation và subgroup là phân tích thứ cấp. Nếu thực hiện kiểm định nhiều giả thuyết xác nhận, dùng hiệu chỉnh Holm cho nhóm đã khai báo; CI trình bày rõ là CI điểm hay CI đồng thời. Không chọn lại budget, seed hoặc metric chính bằng test mới.

Mức tăng có ý nghĩa thực tiễn tạm đề xuất là 0,02 MCC cho cặp meta và 0,01 balanced accuracy cho routing. Đây là biên nghiên cứu do nhóm chọn để tránh theo đuổi mức tăng quá nhỏ, không phải chuẩn ngành. Chốt hoặc điều chỉnh bằng lập luận chi phí và mục tiêu trước khi mở nhãn xác nhận; không điều chỉnh để phù hợp với CI đã thấy.

4.4 Các đóng góp dự kiến

Đóng góp thứ nhất là một thiết kế đối chứng xác định giá trị của đầu ra expert vượt baseline đã khóa, tách nó khỏi tuning ngưỡng và học thêm tầng meta. Đóng góp thứ hai là bằng chứng về tính bổ sung theo lớp, phân biệt phần có sẵn với phần khai thác được trước hoặc sau truy vấn. Đóng góp thứ ba là kết quả xác nhận dưới chuyển dữ liệu, cùng chi phí và gói tái lập có audit context.

Đây là các đóng góp thực nghiệm dự kiến. LR, TF-IDF, Ridge, OOF, các công thức repair/harm và việc dùng MCC là công cụ, không được liệt kê như những phát minh thuật toán. Chỉ viết "kết quả cho thấy" sau khi hoàn thành phép đo tương ứng.

4.5 Giả thuyết chẩn đoán và mức kết luận

Mỗi probe ở mục 6 phải có contrast và hướng hiệu ứng cần kiểm, không viết trước kết quả rằng model không đọc graph hoặc chỉ sao chép nhãn. Có thể khóa một biên tác dụng thực tiễn và hỏi có loại được lợi ích từ biên đó trở lên hay không. Bảng quyết định giữ ba trạng thái: có bằng chứng, chưa đủ bằng chứng, và có bằng chứng tác dụng nhỏ hơn biên đã chọn; không ép mọi giả thuyết thành "ủng hộ hoặc bị bác" khi CI còn rộng.

Các số MCC dưới 0,15, AUROC dưới 0,62 hoặc tỷ lệ flip trên 15 điểm không phải định nghĩa chung của thất bại/ngẫu nhiên/copying. Nếu dùng một con số làm tiêu chí vận hành, phải lý giải và khóa trước, đọc cùng metric/CI và mục tiêu lỗi cụ thể.

5 Thiết kế hệ thống nghiên cứu tối thiểu

5.1 Khóa classifier và feature nền

Tận dụng H3-B: semantic LR, lexical TF-IDF và fusion hiện có là các baseline chính. Giữ encoder frozen và cache; không fine-tune CodeT5/LLM trong bài đầu. Chỉ refit các baseline cần thiết theo cùng protocol dữ liệu, không mở rộng tìm kiếm hàng chục kiến trúc.

Chọn một base classifier và τ bằng train OOF theo quy tắc đã khai báo; giữ nhãn h này cho gate-only, confidence-router và benefit-router. Có thể báo thêm classifier với ngưỡng development-tuned như baseline triển khai mạnh, nhưng phải gắn tên khác. Nếu một policy dùng τ khác thì không coi chênh lệch với gate chung là tác dụng riêng của routing.

34 pre-expert feature có thể tận dụng nguyên thiết kế H3-B: score/logit của các view, score gate, độ lệch giữa view và numeric code features. Không bổ sung thông tin từ response, retry, độ dài output hay latency expert vào router trước truy vấn. Các thông tin này chỉ có sau khi đã trả chi phí.

Trong mã H3-B đã kiểm, workflow.predict ghi gate_score cho nhiều arm. Cột này vẫn là score classifier nền; muốn tính AUC cho stacker hoặc expert phải xuất score riêng có ngữ nghĩa đúng, không đổi nhãn cột gate_score.

5.2 Hai expert prompt cần thiết

E0 code-only. Chỉ có instruction và mã target; không graph, không demo. Nó cho biết năng lực của backbone khi không dùng các khối bổ sung của GRACE. Đây không phải dòng "Zero" có graph trong bài GRACE gốc.

E1 GRACE chuẩn hóa. Cùng target, instruction đầu ra và cấu hình model với E0; bổ sung graph theo một policy cố định và một demonstration lấy từ kho train đã khóa. E0–E1 đo tác động toàn gói graph+demo trong cấu hình này, không tách riêng graph khỏi demonstration.

Giữ giới hạn tổng context theo **token của tokenizer model** trong lượt mới, đồng thời log byte UTF-8 và ký tự để nối với hồ sơ lịch sử. Đề nghị chọn tổng budget đủ giữ code target đầy đủ ở phần lớn cohort, một demo đầy đủ trong phần đã chọn và graph đã đóng gói. Con số budget cụ thể được chọn trên development, không áp lại 8.192 byte chỉ vì run cũ dùng như vậy.

Không cắt im lặng demo tới rỗng. Bộ dựng prompt phải trả trạng thái nội dung từng phần và lý do cắt. Với hàm quá dài không thể giữ trọn, dùng policy xác định trước chỉ dựa trên code, ghi retention và báo kết quả riêng theo độ dài. Không chọn lát cắt nhờ biết nhãn hoặc lời giải lỗ hổng của target. Việc đổi fitter tạo một session mới.

5.3 Đối chứng có và không có expert sau truy vấn

P_pre dùng meta LR với 34 feature nền; P_post dùng cùng meta LR và thêm đúng bốn feature expert đã có trong H3-B. Scaler, C, fold, quy trình OOF, lưới threshold và tiêu chí chọn đều giống nhau. Có thể dành bốn cột bằng 0 cho P_pre để giữ cùng cấu trúc đầu vào trong mã. Cả hai học cùng nhãn development, không một bên được dùng thêm dữ liệu.

P_post phải tính 100% expert query nếu cần g cho mọi target. Nếu P_post thắng P_pre thì có bằng chứng đầu ra expert có giá trị dự đoán tăng thêm trong lớp LR và bộ feature đã chọn; chưa chứng minh router có thể biết trước khi nào nên gọi. Nếu P_post không thắng, CI chưa hẹp có thể chỉ cho thấy chưa đủ bằng chứng.

Với expert invalid, giữ valid flag và raw error. Với router, khóa fallback về nhãn gate chung sau số retry hữu hạn. Với cặp xác nhận P_post–P_pre, P_post phải fallback về chính P_pre đã khóa khi expert không

hợp lệ; nhờ đó hai arm không khác nhau chỉ vì dùng hai fallback khác nhau. Trong meta-fit/OOF, cả hai dùng cùng các dòng development; quy ước expert value 0,5 và valid flag 0 khi invalid được ghi rõ, và prediction P_post ở dòng invalid được thay bằng P_pre ngoài fold tương ứng. Full expert vẫn báo coverage và metric valid-only trên cohort được chỉ rõ; thêm end-to-end result khi dùng cùng fallback nếu cần so ứng dụng. Không so một bên âm thầm bỏ ca lỗi với một bên dự đoán đủ mọi target.

5.4 Routing trước truy vấn

Giữ benefit-router Ridge hiện có làm một baseline; target là +1 khi expert sửa lỗi gate, −1 khi expert làm sai quyết định đúng, 0 khi hai bên cùng đúng hoặc cùng sai. Fit trên development theo OOF, với scaler chỉ fit ở fold học. Đây là lợi ích theo accuracy, chưa trực tiếp tối ưu MCC.

Confidence control ưu tiên mẫu gần biên τ đã khóa, bằng khoảng cách giữa logit(p) và logit(τ), sau khi clip p vào $[10^{-6}; 1-10^{-6}]$. Khoảng cách nhỏ được gọi trước. Không coi score này là xác suất lỗi đã hiệu chuẩn. Giữ uncertainty quanh 0,5 cũ như control lịch sử nếu cần, nhưng không dùng nó làm đối thủ duy nhất khi τ khác 0,5.

Chọn ngẫu nhiên cùng budget là control bắt buộc; lặp 20 seed cố định và báo phân bố. Mọi policy dùng cùng expert response của cùng ID trong một replicate, để khác biệt policy không bị trộn với khác biệt sinh response.

5.5 Một biến thể nhỏ chỉ làm sau khi có chẩn đoán

Nếu phân tích cho thấy target lợi ích theo accuracy thực sự lệch mục tiêu cân bằng lớp, có thể thêm một arm benefit có trọng số lớp lấy từ fold học. Trọng số của mỗi lớp tỷ lệ nghịch tần suất lớp và chuẩn hóa để hai lớp có tổng trọng số bằng nhau. Fit cùng Ridge, không thay feature hoặc lưới tìm kiếm.

Kỳ vọng của target này gần với lợi ích balanced accuracy khi giả định phân bố và trọng số phù hợp, không phải gradient hay tối ưu trực tiếp MCC. Chỉ đưa vào bài đầu nếu đã khóa trước test và còn thời gian; nếu thêm sau khi xem test mới thì gắn nhãn thăm dò và để xác nhận sau. Không làm điều kiện bắt buộc khiến hướng nhanh trở thành một dự án thuật toán mới.

6 Chẩn đoán graph và demonstration bằng các phép can thiệp có kiểm soát

6.1 Vai trò của tái lập có chẩn đoán

Tái lập có chẩn đoán là lớp giải thích cho câu hỏi giá trị bổ sung: khi một nhánh GRACE chưa tạo lợi ích, cần biết đầu vào đã đến model đầy đủ chưa, model nhạy với phần nào và kết quả phụ thuộc thiết kế nào. Khung này được tiếp thu như một thành phần của nghiên cứu chính. Không mở rộng ngay thành sáu câu hỏi độc lập và toàn bộ ma trận hàng trăm nghìn prompt, vì điều đó đi ngược mục tiêu ít thời gian.

Các kết quả lịch sử được giữ làm bằng chứng thăm dò theo từng run. Chỉ các đối chứng mới cùng model, cùng target, cùng kho train, cùng inference và cùng budget/renderer mới dùng để quy tác dụng cho một thành phần. Không gộp ba tên model trong ledger thành một thí nghiệm ba backbone đã kiểm đủ mọi arm.

6.2 Registry arm để tránh nhầm tên giữa các nhánh cũ

Tên mới trong gói mã	Ký hiệu diễn giải	Thành phần được giữ hoặc thay
code_only	B0	Instruction và mã target
graph_only	B1	B0 cộng graph đã khóa
Grace	B2	B1 cộng một demo truy hồi có nhãn train

Tên mới trong gói mã	Ký hiệu diễn giải	Thành phần được giữ hoặc thay
no_graph_in_icl	B2 bỏ graph	Giữ chính demo, nhãn, code, instruction của B2
demo_flip	B2 đảo nhãn	Giữ code demo; chỉ đổi trường nhãn do template sinh
demo_nolabel	B2 bỏ nhãn	Giữ code demo và graph; bỏ dòng/tiêu đề mang nhãn
demo_labelonly	B2 chỉ nhãn	Bỏ code demo; giữ một câu báo nhãn của demo đã chọn
demo_random	B2 random có đối chứng	Thay demo bằng mẫu train cùng nhãn, trong khoảng token đã định
graph_foreign	B1 hoặc B2 graph sai ghép	Thay graph bằng donor train, giữ target và renderer/budget
graph_rewire	B1 hoặc B2 đổi quan hệ	Adapter thay cạnh theo quy tắc đã khóa, giữ tập nút được khai báo
pair và pair_nodiff	Cặp demo	Một vulnerable và một safe; có/không mô tả khác biệt
pair_swap và pair_flip	Probe trên cặp	Đổi thứ tự hoặc đảo đồng thời hai nhãn, giữ các phần còn lại

Các tên B0/B1/B2 ở đây chỉ áp dụng registry nghiên cứu mới. B3 trong lịch sử có thể là security-aware một demo hoặc cặp tương phản ở nhánh khác. Vì vậy, ledger cần lưu cả tên arm đầy đủ, template và commit, không chỉ một mã B3.

6.3 Cặp so sánh đúng cho từng giả thuyết

B0–B1 đo graph khi chưa có demo. B1–B2 đo một demo có nhãn khi đã có graph. B2–no_graph_in_icl đo graph trong ICL. B2–demo_nolabel đo tác động của nhãn được trình bày khi code demo được giữ. demo_nolabel–B1 đo thêm code demo không nhãn trong cùng setup graph; không dùng demo_nolabel–B0 để quy tác dụng riêng cho code demo vì cặp đó còn khác graph.

Demo random lấy toàn kho là một so sánh policy hợp lệ nếu công bố đúng: nó có thể đổi cả nhãn, độ dài và độ khó. Để kiểm phần chọn nội dung, dùng random cùng nhãn demo gốc, cùng renderer và khoảng độ dài token đã khóa. Chọn donor bằng SHA-256 của seed, target ID và tên probe; lưu ID/hash. Random cùng nhãn không bảo đảm mọi độ khó/ngữ nghĩa đều khớp, nên vẫn cần mô tả giới hạn. Chỉ dùng dữ liệu train để cung cấp demo, không chọn donor nhờ nhãn target.

Label-only có độ dài và mức thông tin khác B2; nó là phép chẩn đoán gợi ý nhãn, không phải đối chứng cô lập hoàn hảo nội dung. Không chèn câu vô nghĩa chỉ để lấp số token rồi mặc định câu đó trung tính. Có thể báo hai thiết kế bổ sung: giới hạn ngân sách chung và can thiệp giữ các phần không thay đổi; không buộc mọi prompt có đúng cùng số token khi việc bỏ một thành phần chính là can thiệp.

6.4 Phân biệt các phép thay graph

Graph lấy từ hàm khác cùng số nút gần target được gọi là **graph sai ghép**, không gọi là graph không có thông tin. Nó có thể còn chứa dấu hiệu tương quan, gây mâu thuẫn code–graph hoặc bị model nhận ra rồi bỏ qua. Chọn donor từ train với khoảng số nút, token và cùng renderer; log donor, mức khớp và trường hợp không tìm được donor. Nếu không có donor, ghi thiếu probe thay vì tự đổi tiêu chí từng target.

Đổi thứ tự serialization của node/cạnh có thể giữ nguyên graph về ngữ nghĩa nhưng thay vị trí trong prompt; đó là probe thứ tự, không phải phá thông tin quan hệ. Rewire cạnh là probe khác: adapter phải khai báo tập nút giữ nguyên, loại cạnh, self-loop/cạnh lặp được phép và số cạnh đổi. Một cờ metadata

trong renderer không chứng nhận graph đúng về chương trình; phải kiểm bằng graph source và test của adapter.

Kết quả thay graph làm dự đoán đổi chỉ chứng minh model nhạy với can thiệp, chưa chứng minh sử dụng graph đúng ngữ nghĩa. Kết quả không đổi chưa chứng minh model không đọc graph: code có thể đã đủ, probe yếu, graph bị cắt, hiệu ứng bù trừ hoặc CI quá rộng. Kết luận phải nêu phép thử và effect size cụ thể, kèm audit input.

6.5 Đảo nhãn và kiểm tra lặp

Lưu riêng tỷ lệ bất đồng giữa B2 và demo_flip, tỷ lệ bất đồng giữa hai lượt B2 cùng prompt, và hiệu của hai tỷ lệ. Hiệu có thể âm; không cắt về 0 và không gọi nó là tỷ lệ sao chép nhãn. Hồ sơ cũ ghi 31% là Net Label Sensitivity và 4% là Natural Variance; cần xác minh định nghĩa trước khi tính lại để tránh trừ biến động hai lần.

Đảo nhãn chỉ đổi field label và các câu do template sinh. Không tìm/thay toàn văn vulnerable/safe trong code hoặc comment vì có thể sửa chính đầu vào chương trình. Nếu mô tả khác biệt chứa tên nhãn hoặc chỉ vị trí "ví dụ thứ nhất/thứ hai", pair_flip/pair_swap phải có bản mô tả nhất quán đã kiểm; nếu chưa có thì dùng cặp không mô tả. Gói mã từ chối trường hợp nhập nhằng này.

Min và cộng sự [T20] cùng Wei và cộng sự [T21] cho thấy vai trò nhãn, semantic prior và mapping phụ thuộc thiết kế/model; không cho phép suy ngay rằng mọi LLM chỉ sao chép nhãn. Những kết quả ICL đó là căn cứ đặt probe, không phải đáp án có sẵn cho mã C/C++.

6.6 Quy mô ưu tiên và chấm lỗi gắn với retrieval

Đầu tiên chạy probe trên development đã khóa, khoảng 100–200 target có hai lớp và các mức độ dài; mục đích là kiểm triển khai và chọn một câu hỏi chẩn đoán. Sau đó chỉ đưa một hoặc hai probe có cơ sở vào phép xác nhận được định trước. Pair, H2 và ma trận nhiều budget graph là tầng mở rộng, không phải tất cả đều bắt buộc trước bài đầu.

Có thể mở rộng packet retrieval lên 100 target mỗi dataset nếu đủ thời gian, nhưng giữ thiết kế lấy mẫu, rubric và bất đồng ban đầu. Nối mức liên quan được chấm mù với đúng/sai của prediction như một **mối liên hệ quan sát**; phân tầng theo nhãn target, độ dài, nhãn demo và lượng context. Nhóm có demo liên quan và không liên quan có thể khác độ khó. Không coi tương quan cùng nhãn hoặc cùng rubric với accuracy là bằng chứng nhân quả sao chép.

Kiểm toán nhãn nên lấy ngẫu nhiên trong các nhóm FP/FN đã định, truy patch/CVE/commit hoặc test làm chứng cứ. 50 FP và 50 FN mỗi dataset là một mức mở rộng có thể cân nhắc, không yêu cầu mặc định cộng thêm vào packet 60 ca sửa/làm sai. Báo tỷ lệ nhãn đáng ngờ có điều kiện trong từng nhóm; không suy nó thành tỷ lệ nhãn nhiễu toàn dataset hay trần hiệu năng nếu chưa có thiết kế lấy mẫu và thẩm định thích hợp.

7 Điểm số liên tục và lựa chọn backend

7.1 Mục tiêu và những đại lượng phải tách biệt

Điểm số liên tục cho phép bổ sung AUROC và average precision, kiểm lựa chọn ngưỡng và vẽ đường chất lượng–báo động giả. Nó là phép đo hữu ích khi backend thực sự cung cấp thông tin cần thiết; không phải điều kiện bắt buộc để hoàn thành mọi phân tích từ prediction cũ.

Cần lưu bốn đại lượng khác nhau: verdict được sinh tự do theo contract; argmax giữa hai verbalizer 0/1; score chuẩn hóa giữa hai verbalizer; và nhãn sau ngưỡng chọn trên development. Các đại lượng có thể

khác nhau. Một score trên 0,5 không nhất thiết trùng verdict model nếu token có xác suất lớn nhất trong toàn vocabulary là token khác.

7.2 Tính score bằng candidate thật

Tại cùng prefix đã serialize bằng chat template khóa, nếu hai label là một token ổn định, gọi logprob của chúng là lp0 và lp1. Score bằng sigmoid của lp1 trừ lp0, tương đương p1 chia p0+p1. Đây là xác suất điều kiện trên hai token nhãn, chưa được hiệu chuẩn thành xác suất mã có lỗ hổng. Khi có logits toàn vocabulary trước masking, báo thêm label mass p0+p1.

Tokenizer phải được pin và kiểm tại **đúng prefix có generation cue**; encode riêng chuỗi "0" hoặc "1" chưa đủ. Token "1", " 1" và xuống dòng rồi "1" không phải mặc nhiên cùng một event. Nếu verbalizer nhiều token, phải định nghĩa sequence likelihood/length normalization riêng hoặc dừng scoring; không lấy token đầu rồi giữ nguyên tên metric.

Nếu top-k không chứa đủ cả hai candidate, score phải null với trạng thái thiếu, dù verdict có thể hợp lệ. Không tự điền −30, không dùng điều kiện "có ít nhất một nhãn", không strip chuỗi token rồi để các biến thể ghi đè nhau. Gói mã cung cấp scorer strict cho đủ hai logprob và scorer từ full raw logits; nó không khôi phục xác suất từ thông tin backend chưa trả.

7.3 Ba đường triển khai và kiểm tra năng lực

Đường ít thay đổi nhất là giữ backend hiện có nếu nó xuất logprob đúng candidate/vị trí và cho phép lưu đầy đủ contract. Đường thứ hai là dùng Transformers cục bộ để lấy full next-token logits. Đường thứ ba là vLLM tự host, thuận lợi cho batching, pin model và quan sát tài nguyên. Không bắt đầu bằng thuê GPU nếu câu hỏi hiện tại có thể trả lời bằng artifact offline.

Qwen2.5-Coder-32B-Instruct-AWQ là một cấu hình có thể pilot, không phải lựa chọn mặc định bảo đảm vừa GPU hoặc nhanh nhất [T19]. Với 32,5 tỷ tham số, riêng trọng số BF16 xấp xỉ 65 GB theo byte thập phân trước overhead; AWQ giảm phần lưu trọng số nhưng vẫn cần KV cache, activations, runtime và workspace. GPU 24/48/80 GB chỉ là nhóm cấu hình để thử theo batch/context cụ thể, không phải bảng bảo đảm fit. Nếu hạ xuống model 14B/7B, ghi là đổi model, không coi như cùng expert rẻ hơn.

vLLM không bảo đảm tái lập mặc định. Tùy online/offline và phiên bản, cần kiểm scheduling/batch invariance; ngay cả khi đó giới hạn hardware/version vẫn quan trọng [T17]. Pin model revision, tokenizer, quantization, CUDA/driver, engine, chat template, concurrency và seed. Self-host vẫn có thể OOM, lỗi server hoặc transport; vẫn phải giữ error/invalid và repeat thực đo.

7.4 Constrained decoding và điểm số raw

Ép đầu ra thuộc 0/1 có thể làm verdict sạch hơn, nhưng thay đổi contract sinh và có thể đổi cách chuẩn hóa logprob. Nó không phải cách tự động bù candidate bị thiếu trong top-k. Với các phiên bản vLLM hiện hành, giao diện structured_outputs thay cho các trường guided_* đã loại bỏ; cần pin version và kiểm tài liệu tương ứng [T18, T22].

Nếu dùng constrained verdict, lưu nó thành arm/protocol riêng; score phục vụ so sánh ranking cần ghi raw trước constraint hay đã qua masking. Không gắn label mass của phân phối bị mask thành khối lượng hai nhãn trong vocabulary gốc. Bản gói mã mặc định không bật constrained decoding để che ca thiếu score.

7.5 Ngưỡng và metric không ngưỡng

Báo AUROC cùng average precision theo định nghĩa average_precision_score, nêu rõ không đồng nhất mọi cách tính diện tích PR hình thang với AP. Đọc AP cùng prevalence của cohort; trên cohort phân tầng

phải dùng trọng số nếu suy về eligible population. Score coverage được báo riêng với verdict coverage và dùng cùng score-complete ID khi so các arm.

Mỗi score có thể được báo tại ngưỡng 0,5 và tại ngưỡng chọn trên development, nhưng hai chế độ trả lời hai câu hỏi khác nhau. Khi so policy được tuning riêng, gọi là so toàn policy. Khi quy tác dụng riêng của expert/router, dùng cùng gate và cùng quy trình selection như cặp chính. Không coi ngưỡng validation đã tối ưu là điểm tốt nhất tuyệt đối trên population mới.

Hai arm có cùng AUC vẫn có thể khác thứ hạng, đường ROC giao nhau và sửa/làm sai những mẫu khác nhau. Dùng thêm TPR/FPR, confusion, rank/score agreement và CI để đọc đánh đổi; AUC không giải quyết dứt điểm câu hỏi cơ chế. Tương tự, AUROC 0,62 không phải tự động là ngẫu nhiên 0,5.

7.6 Pilot tài nguyên trước khi lập dự toán

Chạy 20 prompt để kiểm schema/candidate, sau đó 100 prompt development đủ các mức độ dài để đo token, thời gian, throughput và peak memory. Warm-up được tách khỏi đo chính; đo prefill/input tokens, output tokens và full request latency riêng. Không dùng output tokens/giây trong benchmark khác để suy prefill của prompt dài [T23].

Dự toán thời gian gồm load/warm-up, tổng input/output token chia throughput tương ứng trong cấu hình đã đo, overhead, retry, scoring và lưu artifact. Ví dụ 120.000 prompt nhân 2.500 input token là khoảng 300 triệu input token; giả định 10.000 input token/giây cho 8,33 giờ thuần tính nhưng chưa chứng minh đạt throughput đó. Phần này chỉ là công thức khối lượng. Giá GPU/giờ được lấy tại ngày lập dự toán; không cố định tổng tiền từ một giả định chưa đo.

8 Đo tính bổ sung và chi phí một cách đúng nghĩa

8.1 Bốn trạng thái đúng sai và phân rã theo lớp

Với mỗi target có hai prediction hợp lệ, gắn một trong bốn trạng thái: cả hai đúng; chỉ classifier đúng; chỉ expert đúng; cả hai sai. Tách bảng theo nhãn y=0 và y=1 trước khi cộng tổng.

Nhãn thật	Expert sửa lỗi classifier	Expert tạo lỗi mới	Diễn giải bảo mật
y=1	FN của classifier thành TP	TP của classifier thành FN	Bắt thêm hoặc bỏ sót thêm mẫu vulnerable
y=0	FP của classifier thành TN	TN của classifier thành FP	Giảm hoặc tạo thêm báo động giả

Trên tập S được route, ký hiệu R_y(S) và H_y(S) là số sửa đúng/làm sai ở lớp y. Khi mỗi mẫu có cùng trọng số, thay đổi accuracy trên toàn cohort bằng tổng sửa đúng trừ tổng làm sai, chia N. Thay đổi balanced accuracy bằng một nửa tổng của (R_1−H_1)/N_1 và (R_0−H_0)/N_0. Phải dùng N_0, N_1 của toàn cohort, không dùng số mẫu được route riêng từng lớp làm mẫu số.

MCC không phân rã theo tổng lợi ích từng mẫu. Với mỗi policy, ghép nhãn cuối cho toàn cohort rồi tính lại confusion matrix và MCC. Khi dùng mẫu phân tầng có trọng số, thay số đếm bằng số đếm có trọng số tương ứng và dùng đúng thiết kế lấy mẫu.

8.2 Ba mức tính bổ sung cần phân biệt

Bổ sung có sẵn là những trường hợp expert đúng khi classifier sai. Oracle chọn bên đúng theo nhãn thật cho biết có thông tin bổ sung để khai thác về mặt accuracy, nhưng không nói có học được quy tắc chọn đó từ feature hay không.

Bổ sung khai thác sau truy vấn được kiểm bằng P_post so P_pre. Nó trả lời liệu đầu ra expert mang thông tin tăng thêm sau khi đã biết mọi feature nền và đã trả chi phí query.

Bổ sung khai thác trước truy vấn được kiểm bằng router so random/confidence và gate chung ở cùng budget. Có P_post tốt hơn chưa đủ để có router tốt: tín hiệu có thể chỉ xuất hiện sau khi đọc response. Đây là một kết quả khoa học có ý nghĩa dù router không thắng.

Oracle với ngân sách "không quá k query" chỉ chọn những ca sửa đúng; có thể dùng ít hơn k. Không ép oracle gọi các ca gây hại cho đủ quota. Oracle không dùng để chọn policy hoặc threshold thực tế. Không gọi MCC của một oracle tối ưu accuracy là MCC tối ưu khả dĩ.

8.3 Đường chất lượng theo ngân sách truy vấn

Dùng q thuộc {0%; 5%; 10%; 25%; 100%}. Trong nghiên cứu batch, policy chọn trước top ceil(qN) target theo score pre-expert; phá hòa bằng hash ID cố định. Cách này không dùng nhãn test và tạo cùng số query giữa các policy, nhưng cần gọi đúng là **batch routing**. Policy chính tại 10% phải được định trước.

Khi xét triển khai online, dùng cutoff khóa trên development; tỷ lệ query trên test có thể khác q. Không thay cutoff theo nhãn test để vừa đạt quality cao vừa trông như đúng budget. Báo hai setup riêng nếu thực hiện cả hai.

Trên cohort phân tầng, top ceil(qN) chỉ bảo đảm cùng q theo số target của cohort chẩn đoán. Phải báo thêm tỷ lệ query có trọng số bằng tổng trọng số các target được route chia tổng trọng số cohort. Không dùng trọng số phụ thuộc nhãn để chọn routing mask. Các đường metric có trọng số không được gọi là cùng budget quần thể nếu tỷ lệ query có trọng số khác nhau. Phân tích triển khai ở prevalence tự nhiên dùng cutoff đã khóa từ nguồn development và báo query ratio thực tế.

Mỗi điểm q báo MCC, BA, F1, TP/FP/FN/TN, coverage và chi phí. "Số sửa ròng trên 100 query" là mô tả dễ hiểu nhưng dùng mục tiêu accuracy; phải kèm số FN và FP ròng. Không chọn điểm đẹp nhất trên test làm headline.

8.4 Định nghĩa chi phí

Tách ba nhóm: chi phí tạo đặc trưng/học/chọn policy; chi phí suy luận theo thuật toán; và chi phí vật lý của chiến dịch thực nghiệm. Học meta cần response development; benchmark cần full expert reference; cả hai có thể tốn nhiều hơn policy được triển khai cuối.

Query ratio 0 không có nghĩa tổng chi phí 0: classifier cần embedding, preprocessing, TF-IDF và suy luận. Ngược lại, vài request mới sau cache không có nghĩa thuật toán chỉ cần vài request. Báo cold run và warm-cache run riêng, số target yêu cầu expert, request thực, retry, cache hit, input/output token, thời gian và tiền theo bảng giá tại thời điểm chạy.

Nếu chưa có usage token, ghi missing. Không suy chi phí tiền từ số byte hoặc query ratio. Nếu đo latency, cố định concurrency và phần cứng, báo median/P95 cùng số mẫu; không so một bên chạy song song cao với một bên tuần tự rồi quy chênh lệch cho phương pháp.

9 Dữ liệu và thiết kế xác nhận

9.1 Hai lớp bằng chứng

Lớp A dùng Devign/ReVeal và artifact hiện có để thăm dò, sửa thiết kế và định hình giả thuyết. Tính lại full cohort và mask không trùng exact cho mọi arm bằng cùng ID. Không refit hoặc chọn lại winner dựa trên mask. Kết quả trên mask được gọi là phân tích độ nhạy.

Lớp B dùng cohort ngoài chưa được nhóm sử dụng để chọn phương án, khóa trước inference và chấm. PrimeVul là lựa chọn ưu tiên vì có dữ liệu và metadata phục vụ đánh giá nghiêm ngặt [T8]. Tuy vậy, PrimeVul tổng hợp nhiều nguồn; tên dataset khác không bảo đảm mẫu khác. Phải kiểm overlap với toàn bộ train, development, retrieval bank và test Devign/ReVeal đã tiếp xúc.

9.2 Lập cohort ngoài theo quy trình tiết kiệm

Chọn một release/commit PrimeVul và ghi checksum. Dùng official test làm nguồn eligible, giữ metadata project/commit/CVE nếu có. Lọc exact hash và normalized-code hash với mọi nguồn cũ; phát hiện near-clone bằng một quy tắc token cố định đã chọn trên development. Không dùng quy tắc xóa bừa literal/identifier làm tiêu chuẩn duy nhất, vì có thể xóa chính khác biệt bảo mật. Ghi số bị loại theo từng bước và kiểm thủ công một mẫu gần ngưỡng clone.

Mục tiêu thực dụng là lấy ngẫu nhiên trong từng lớp **200 vulnerable và 1.800 safe** từ tập eligible, hoặc toàn bộ vulnerable nếu không đủ 200. Đây là cohort chẩn đoán phân tầng, không có tỷ lệ tự nhiên của PrimeVul. Không thêm quota project trong lượt đầu nếu chưa xây sampling weights tương ứng. Khóa seed lấy mẫu độc lập với seed huấn luyện; không loại ca dài, parse khó hoặc thiếu graph sau khi thấy output.

Để suy hiệu năng về eligible population, lưu N_y là số mẫu eligible của lớp y và n_y là số được chọn. Mỗi mẫu lớp y có trọng số N_y/n_y. Cộng confusion matrix có trọng số rồi tính metric. Báo cả metric không trọng số cho cohort chẩn đoán và metric có trọng số cho tỷ lệ tự nhiên của population đã định nghĩa. Không gọi metric 10% vulnerable là hiệu năng trực tiếp trên PrimeVul tự nhiên.

Khoảng tin cậy phải tôn trọng thiết kế phân tầng và phụ thuộc nhóm; dùng bootstrap/resampling có trọng số theo thiết kế đã mô tả, hoặc báo riêng CI phân tầng và độ nhạy theo nhóm khi không đủ hỗ trợ cho một ước lượng kết hợp. Không lấy bootstrap hàm không trọng số rồi gắn vào metric population có trọng số. Với cặp vulnerable/fixed, giữ liên kết pair/commit khi suy luận; phép đánh giá pair cần có đủ cả hai thành viên, được báo thành một cohort riêng.

9.3 Xác nhận chuyển dữ liệu và cách tránh chọn nguồn hậu nghiệm

Đường nhanh nhất là **cross-dataset transfer**: khóa classifier học trên ReVeal train làm mô hình nguồn chính; dùng development ReVeal để học/chọn P_{pre}, P_{post} và router; sau đó áp nguyên sang cohort PrimeVul đã khóa. Chọn nguồn ReVeal trước khi chấm cohort ngoài, không chọn giữa nguồn Devign/ReVeal bằng điểm PrimeVul. Nguồn Devign chỉ làm phân tích thứ cấp nếu đủ thời gian.

Như vậy, kết quả ngoài trả lời về tính khái quát dưới chuyển dữ liệu, không phải chất lượng in-distribution của một classifier được học trên PrimeVul. Nếu lợi ích biến mất, có thể do domain shift của gate, router hoặc expert; cần tách giải thích. Huấn luyện thêm trên PrimeVul train là thí nghiệm khác, thuộc bước mở rộng, không tráo vào headline để cứu kết quả chuyển miền.

"Chưa được nhóm dùng để chọn phương án" không có nghĩa LLM chắc chắn chưa nhìn thấy mã trong pretraining. Ghi ngày commit và hạn chế contamination; không tuyên bố hết rò rỉ chỉ bằng khử exact duplicate. Nếu nhóm đã dùng phần test PrimeVul này trước đây, cần chọn cohort khác còn phù hợp thay vì gọi nó là mới.

9.4 Chuẩn hóa expert cho lượt mới

Các tên model trong [N2–N3] là tên khai báo của run lịch sử. Trước lượt mới, kiểm tra endpoint thực sự dùng được, model/version được provider hỗ trợ, khả năng kiểm soát thinking, tokenizer và logging. Không mặc định các endpoint lịch sử còn hoạt động và không chọn một model đời cũ chỉ vì từng được đề xuất.

Giữ cùng provider, model/version nếu có, instruction, parameters, retry và parser cho expert development ReVeal và cohort ngoài. Nếu thay endpoint hoặc prompt packing, tạo lại response development cần cho meta-fit; không ghép meta học từ expert cũ với test expert mới thành một phép xác nhận ổn định.

Để nhắm bài full hội thảo, ưu tiên thêm một backbone khác có năng lực và cấu hình phục vụ xác định được. Chạy hai prompt E0/E1 trên cùng cohort ngoài nếu ngân sách cho phép; ít nhất cần một phép lặp đã khóa trên cohort đủ số positive. Một backbone thứ hai chỉ chạy vài chục ví dụ đẹp không đủ tạo kết luận khái quát. Nếu chưa làm được, giới hạn bài theo một expert hoặc chọn track ngắn/tái lập phù hợp.

9.5 Chuẩn hóa code và thay bank là hai việc khác

Giữ raw SHA-256 để định danh bytes của code canonical; normalized hash là trường phụ phục vụ phát hiện trùng theo một phép biến đổi đã khai báo. Xóa comment/khoảng trắng bằng regex có thể thay nội dung literal hoặc token trong C/C++; cần lexer phù hợp và test trên string, character literal, macro, comment. Không băm snippet hay chuẩn hóa mất dấu hiệu bảo mật rồi gọi là exact duplicate.

Jaccard token từ 0,9 hoặc near-neighbor top-50 là quy tắc dò clone gần đúng, không bảo đảm tìm hết clone. Khóa ngưỡng bằng development, lưu candidate generator và kiểm vài ca gần biên. Nếu loại mẫu khỏi train/retrieval bank theo overlap policy mới, phải tạo dataset/session/cache identity mới; refit các thành phần bị ảnh hưởng và retrieve lại. Ngược lại, clean-mask sensitivity trên prediction cũ chỉ đổi phần chấm và không refit. Không trộn hai loại thí nghiệm vào cùng một bảng như chỉ replay.

10 Ma trận thực nghiệm tối thiểu

10.1 Những nhánh cần giữ và những nhánh có thể bỏ

Nhánh	Câu hỏi trả lời	Có cần thêm API khi đã có E1 đầy đủ
Semantic LR, lexical, fusion	Baseline nhẹ có đủ mạnh không	Không
Gate chung và gate tuned độc lập	Tác dụng classifier và threshold	Không
E0 code-only	Năng lực backbone	Có
E1 GRACE chuẩn hóa	Tác dụng toàn gói graph và demo	Có hoặc reuse đúng contract
P_pre và P_post LR	Giá trị riêng của expert output	Không
Random, confidence, benefit router	Có chọn được query hữu ích không	Không
Always safe và always vulnerable	Mốc kiểm tra metric	Không

Không cần tiếp tục stack MLP, toàn bộ loss H1, direct-high cascade hay toàn bộ H2 trong vòng đầu. Một static analyzer nhẹ có thể thêm làm mốc ngoài nếu chạy được trên code fragment, nhưng phải báo parse coverage và phạm vi rule. Nó không thay thế classifier baseline; nếu nghiên cứu chuyển sang whole-project hoặc công cụ ứng dụng, static analyzer trở thành đối chứng quan trọng hơn.

10.2 Các phép đo theo từng câu hỏi

Câu hỏi	Cặp hoặc phân tích	Kết quả phải xuất
RQ1	P_post so P_pre; E1, E0 và gate làm mốc	Paired ΔMCC, ΔBA, F1; CI; invalid; chi phí
RQ2	Bốn trạng thái đúng/sai, tách lớp	FN sửa/tạo, FP sửa/tạo; case packet; đặc điểm context
RQ3	Benefit so confidence/random tại q cố định	Đường chất lượng–query; số sửa/làm sai; oracle accuracy chẩn đoán
RQ4	Full/mask, seed, repeat, cohort ngoài	Độ bền thứ hạng, effect size theo cohort; tổng chi phí

10.3 Kiểm soát graph và demonstration với chi phí nhỏ

Audit toàn bộ prompt tự động trước khi gọi API: target retention, số token code demo thực nhận, demo label, số node/cạnh graph, tỷ lệ bị cắt, hash và thứ tự. Xem thủ công 30 prompt development chọn phân tầng theo độ dài và retention. Đây là kiểm tính toàn vẹn đầu vào, không phải chọn ví dụ để đánh giá model.

Nếu cần giải thích E0–E1, chỉ bổ sung một can thiệp nhỏ đã định trước trên development: giữ target/demo/instruction, bỏ graph và không lấp chỗ trống. Phép này đo graph khi đã có demo. Muốn đo riêng demo, giữ graph và bỏ demo. Không bắt buộc chạy một ma trận đầy đủ cho cả hai model ở bài đầu.

Nếu nghi nhãn demo ảnh hưởng mạnh, label-flip chỉ là phép chẩn đoán độ nhạy với nhãn trình bày, cần repeat original làm mốc. Nó không đo tỷ lệ "chép nhãn". Không dùng kết quả label-flip đơn lẻ để giải thích mọi sai khác hoặc để tuyên bố model không đọc code.

10.4 Phân tầng thay cho ma trận lớn mặc định

Tầng lõi là replay offline và cặp P_pre/P_post, gate/full expert, random/confidence/benefit, E0/E1 trên cohort xác nhận. Tầng chẩn đoán thêm B1 và một số probe ở development; mở rộng chỉ probe phục vụ kết luận chính. Tầng sau mới gồm H2 đầy đủ, nhiều graph budget, pair-swap/flip/no-diff trên full cohort, nhiều model và dataset. Bảng arm là menu câu hỏi, không phải danh sách phải chạy tất cả. Một ma trận khoảng 120.000 prompt chỉ hợp lý sau khi pilot xác nhận ngân sách và tính cần thiết.

11 Quy trình thực nghiệm và cập nhật phần mềm theo từng bước

Các bước dưới đây theo thứ tự phụ thuộc. Mỗi bước có mục tiêu, lý do, thao tác và đầu ra. Gói đi kèm GRACE_research_kit.zip cung cấp các mô-đun độc lập và fixture kỹ thuật để thử luồng; fixture không phải kết quả nghiên cứu. Không cần hoàn thành toàn bộ probe mở rộng trước khi chạy cặp xác nhận chính.

E01 Tạo bản làm việc và lưu identity

Mục tiêu và ý nghĩa. Bảo vệ run lịch sử và có một trạng thái mã tái tạo được; đây là điều kiện để biết thay đổi nào gây ra chênh lệch. Tạo branch hoặc thư mục nghiên cứu mới từ commit đang dùng, lưu environment và checksum; không sửa winner lock, raw hoặc nhãn gốc.

Thao tác và đầu ra. Giải nén gói mã vào thư mục research của project, cài requirements trong môi trường riêng và chạy unittest. Ghi commit nguồn, model/tokenizer revision, engine và template vào protocol.template.json. Hàm validate_protocol cố ý từ chối template chưa khóa hoặc còn trường thiếu. Mã không tự đăng OSF, push GitHub, gửi bài hay thuê tài nguyên.

E02 Chuẩn hóa prediction và cohort bằng adapter

Mục tiêu và lý do. Loại sai join, hash và score trước khi diễn giải hiệu năng. Đọc canonical manifest chứa ID, nhãn chuẩn, full code hash, group và sampling weight; dùng export_canonical để ghép một arm. Nhãn target chỉ được ghép ở phía đánh giá.

Đầu ra và kiểm tra. Canonical JSONL phải có đủ một dòng/target; từ chối thiếu/thừa/duplicate ID, code hash hay nhãn mâu thuẫn. Không băm snippet 120 ký tự để thay hash code đầy đủ. Xuất summarize cho mọi arm và giữ coverage/invalid/error. Score chưa xác định ý nghĩa phải null; gate_score không được tự gán cho expert/stacker.

E03 Tạo mask và bảng sửa lỗi làm sai offline

Mục tiêu và ý nghĩa. Xác định expert có dư địa bổ sung nào trước khi phát sinh chi phí mới. Tạo mask không trùng theo hash giữa test cũ và dữ liệu học/chọn; áp cùng mask lên mọi arm, không refit hoặc chọn lại winner.

Chạy repair-harm trên full và clean sensitivity; lưu ID của bốn trạng thái theo nhãn, oracle accuracy chẩn đoán và confusion. Nếu tổng sửa/làm sai không khớp các confusion thì dừng kiểm join. Hoàn thành bằng bảng có cả số đếm, tỷ lệ và nguồn run; không gọi mask mới là holdout mới.

E04 Xuất thành phần prompt và audit lượng thông tin thực nhận

Mục tiêu và lý do. Kiểm tra model đã nhận phần nào trước khi giải thích retrieval/graph. Xuất target, demo và graph từ cùng selection đã khóa trước fitter; nếu graph đã là prefix thì phải truy graph nguồn để thử full/H2.

Dùng build_prompt với tokenizer/chat-template counter thật, max prompt token và output reserve. Overflow hoặc demo rỗng phải báo lỗi hoặc đi policy cắt đã khai báo, không xóa âm thầm. Audit hashes các khối, byte/token, retained source và exact prompt. Đọc thủ công 20–30 prompt theo độ dài; đầu ra là prompt_components.jsonl, prompt bundle và báo cáo integrity.

E05 Khóa một giả thuyết chẩn đoán và kiểm probe

Mục tiêu và ý nghĩa. Tách phần nhãn, code demo và graph mà không thay nhiều biến ngoài dự kiến. Chọn một hoặc hai probe ở mục 6 trên development; giữ registry tên arm và donor manifest.

Chạy audit_component_change để diff chỉ ở demos, graph hoặc difference đúng can thiệp. Unit test kiểm bất biến khi thay/bỏ field nhãn target ở adapter; không quét cấm ký tự 0/1 trong code. Demo random phải dùng kho train đầy đủ, cùng nhãn demo gốc khi đo nội dung. Đầu ra là probe specification, seed và danh sách phần chưa cô lập hoàn toàn; không chọn lại probe bằng holdout mới.

E06 Kiểm backend và score contract

Mục tiêu và lý do. Đảm bảo số liên tục là một phép đo thật, không phải kết quả do bù logprob thiếu. Kiểm 20 prompt về candidate tokenization, raw verdict, logprob đầy đủ, label mass nếu có và finish reason. Với single-step contract đã sinh đủ 0/1, finish_reason length không tự làm verdict sai.

Nếu API chỉ có nhãn, tiếp tục phân tích nhãn và ghi score null. Nếu cần score, dùng backend có đủ candidate hoặc script score_local_hf.py trên model local đã pin; không tự tải model hay gọi API trong kiểm thử fixture. Pilot 100 prompt đo tài nguyên trước full run. Đầu ra gồm capability report, engine/model/tokenizer manifest và dự toán cập nhật.

E07 Khóa dữ liệu xác nhận và protocol

Mục tiêu và ý nghĩa. Tạo phép kiểm mới thực sự ngoài lựa chọn đã có. Giữ dữ liệu cũ là exploratory; tạo cohort ngoài, overlap audit, sampling weights và group đúng mục 9. Nếu nhóm đã dùng cohort ngoài đó để chọn phương án thì phải chọn cohort khác.

Khóa primary contrast, metric, δ, hai seed loại khác nhau cho sampling/training, quy tắc bootstrap, budget, endpoint, parser và chi phí. Commit/tag có timestamp lưu cục bộ là đủ để bắt đầu kiểm soát phiên bản; đăng OSF là tùy chọn nếu có chủ đích chia sẻ. Đầu ra là protocol đầy đủ đã được validator chấp nhận, cùng disclosure tiếp xúc test; không dùng từ preregistration cho phân tích lịch sử.

E08 Fit cặp meta và router cùng điều kiện

Mục tiêu và lý do. Đo giá trị riêng của expert thay vì lợi ích học thêm tầng LR hoặc thay ngưỡng. Khóa một gate/τ; dùng pre_features chung, cùng development và group folds cho P_pre/P_post.

Trước meta-fit phải có expert development đúng contract: nếu endpoint, tokenizer hoặc packing đổi, chạy và khóa response development theo cơ chế inference ở E09 trước; không tái sử dụng expert cũ không khớp. Hàm fit_matched tạo OOF cho LR pre/post, chọn threshold theo cùng lưới rồi refit; không nhận test label làm đầu vào fit. P_post expert-invalid phải dùng nhãn P_pre tương ứng, gồm cả OOF và suy luận; router fallback gate. Xuất folds, OOF, threshold và model checksum; điểm OOF đã dùng chọn vẫn không phải xác nhận ngoài. Chạy ba seed theo protocol, không chọn seed tốt nhất.

E09 Inference theo session mới và exact request cache

Mục tiêu và ý nghĩa. Tạo response development/test có cùng expert và giữ được truy vết ca lỗi. Khóa prompt bundle, request identity chứa model/tokenizer/engine/config/cohort/template, không chỉ prompt hash. Tách request error, parse-invalid và fallback.

Phần expert development cần cho meta đã được tạo trước E08 theo cùng cơ chế này. Chỉ sau khi meta-model, threshold và router đã khóa mới chạy E0/E1 trên cohort xác nhận theo ngân sách đã định; checkpoint chỉ reuse exact identity. Giữ raw, usage, attempts và trạng thái score/verdict riêng. Một request lỗi sau hết retry vẫn có dòng error. Việc lấy full expert reference cho benchmark không phải query ratio của router triển khai.

E10 Replay ngân sách và tính thống kê

Mục tiêu và lý do. Đánh giá việc chọn đúng query và độ bất định trên cùng target/response. Replay random, confidence và benefit tại q đã khóa; mọi mask chỉ dùng pre-query feature. q trên cohort phân tầng là q của sample, cần báo thêm weighted query ratio.

Chạy summarize, compare và repair-harm qua CLI; observed delta là metric B trừ A trên cohort gốc, không phải trung bình bootstrap. CI phân tầng và cluster sensitivity được gắn tên; module không tự cung cấp một phương sai survey tổng quát. Các replicate MCC/AUC không xác định được đếm và cảnh báo; không loại âm thầm. Đầu ra JSON đủ truy vết để sinh bảng CSV/LaTeX và hình.

E11 Repeat expert và thẩm định lỗi theo mã nguồn

Mục tiêu và ý nghĩa. Tách dao động sinh response khỏi bất định dữ liệu và kiểm giải thích kỹ thuật. Repeat cùng prompt trên tập đã chọn trước, giữ exact cache bypass có chủ đích và lưu replicate ID; không reuse cache rồi coi như đã lặp độc lập.

Lấy packet sửa/tạo FN/FP theo kế hoạch, chấm độc lập, ẩn arm và giữ bất đồng trước hòa giải. Có thể nối retrieval relevance với outcome nhưng phải gắn là association. Kiểm nhãn bằng nguồn gốc

patch/commit; không sửa label vì lời giải thích của LLM. Đầu ra là repeat report, packet, agreement và bảng sensitivity nhãn có chứng cứ.

E12 Sinh bài và gói tái lập

Mục tiêu và ý nghĩa. Biến kết quả thành bằng chứng có thể kiểm tra, kể cả khi kết quả âm. Mỗi RQ cần câu trả lời, effect size, CI, scope và chi phí; không ép kết luận theo giả thuyết ban đầu.

Sinh bảng/hình bằng script, đối chiếu toàn bộ arm đã khóa, giữ cả kết quả bất lợi và thiếu dữ liệu. Gói release gồm schema, manifest, prompt, raw/prediction được phép chia sẻ, score thật, model/fold metadata và README. Có thể lưu bản archive DOI trên Zenodo/Figshare khi quyết định công bố; không cam kết mọi artifact dưới 5 GB hoặc chạy lại trong một ngày GPU trước khi đo. PROGRESS.md cập nhật theo milestone với việc xong, vướng mắc và link artifact, không cần phân vai cá nhân trong đề cương.

12 Thống kê và tiêu chí kết luận

12.1 Các chỉ số và quy ước

MCC là metric chính cho cặp RQ1, BA và confusion matrix giải thích cân bằng lớp; F1 hỗ trợ đối chiếu literature. Báo positive rate, precision/recall, coverage, invalid và fallback theo lớp. Với predictor hằng, MCC theo công thức có mẫu số bằng 0: ghi N/A và giải thích không có năng lực phân biệt; nếu thư viện trả 0 theo quy ước thì ghi rõ riêng, không trộn hai quy ước. Precision không có predicted positive cũng ghi N/A.

Các metric trên cohort phân tầng phải gắn nhãn unweighted/weighted và population tương ứng. PR-AUC/ROC-AUC chỉ áp dụng predictor có score liên tục; một nhãn expert 0/1 không tạo ra đường ROC đầy đủ. Không yêu cầu model tự báo confidence rồi mặc định nó đã hiệu chuẩn.

12.2 Khoảng tin cậy và kiểm định

Đề nghị 5.000 lượt paired resampling cho cặp chính và seed bootstrap cố định; lưu chỉ số mẫu/group đã rút để tái lập. Lượt bootstrap mất một lớp hoặc mẫu số MCC bằng 0 phải được ghi nhận; không âm thầm loại chúng rồi báo số lượt nhỏ hơn mà không giải thích. Với ít positive/group, cần xem lại cỡ mẫu và CI phù hợp thay vì ép có một khoảng đẹp.

McNemar dùng bảng đúng/sai của hai phương pháp theo nhãn thật, hai ô bất đồng là A đúng/B sai và A sai/B đúng [T14]. Nó trả lời khác biệt tỷ lệ lỗi, không trả lời trực tiếp khác biệt MCC/F1. Vì vậy ReVeal có thể tăng MCC rõ nhưng McNemar không có ý nghĩa thống kê; điều đó không tự mâu thuẫn. Exact McNemar chỉ dùng số đếm cặp thô, không đưa confusion có trọng số vào phép nhị thức. Trên cohort lấy cố định số mẫu theo lớp hoặc có phụ thuộc nhóm, xem McNemar như phân tích phụ mô tả cohort; suy luận population chính dựa vào CI ghép cặp phù hợp thiết kế lấy mẫu và nhóm.

Độ bất định do lấy mẫu, biến động seed huấn luyện và biến động response expert là ba nguồn khác nhau. Báo riêng CI dữ liệu của replicate chính, kết quả từng seed và kiểm repeat expert; không cộng tất cả thành một số ± chung khi chưa có thiết kế phân cấp phù hợp.

12.3 Phân biệt chưa đủ bằng chứng và lợi ích quá nhỏ

Kết quả cặp xác nhận	Cách kết luận phù hợp
CI của Δ nằm hoàn toàn trên 0 và mức tăng đủ ý nghĩa thực tiễn	Có bằng chứng lợi ích trong setup đã kiểm
CI chứa 0 và còn bao gồm mức tăng đáng quan tâm	Chưa đủ bằng chứng; không kết luận tương đương
Cận trên CI một phía đã chọn trước nhỏ hơn biên lợi ích δ	Có bằng chứng loại được lợi ích ít nhất δ trong setup này
CI nằm dưới 0	Có bằng chứng phương án bất lợi theo metric đang xét

Để đưa ra kết luận "không đáng dùng", cần kết hợp cận lợi ích với chi phí và mục tiêu bỏ sót/báo động giả; không chỉ dựa vào p>0,05. Nếu muốn dùng phép thử tương đương chính thức, phải khóa hai biên và phương pháp từ trước; không thay bằng một CI chứa 0.

12.4 Cỡ mẫu và sức mạnh bằng chứng

200 positive và 1.800 negative là mục tiêu cân bằng chi phí, không bảo đảm đủ lực cho ΔMCC 0,02. Trước xác nhận, dùng cấu trúc lỗi development để mô phỏng các mức hiệu ứng và ước lượng độ rộng CI dự kiến; chỉ dùng dữ liệu development cho việc này. Ghi kịch bản số ca bất đồng và tỷ lệ lớp, không đưa một công thức cỡ mẫu chung cho MCC.

Nếu dự toán cho thấy không thể có CI đủ hẹp trong ngân sách, khóa mục tiêu là ước lượng hiệu ứng với độ bất định công khai và viết bài thăm dò/tái lập phù hợp. Không chia nhỏ test rồi kéo dài thu thập sau mỗi lần thấy p-value cho đến khi có ý nghĩa. Muốn thiết kế tuần tự phải khai báo quy tắc và hiệu chỉnh trước.

12.5 Ràng buộc của mô-đun thống kê đi kèm

Gói mã tính observed delta và CI riêng, tách cohort prediction-valid với score-complete. Paired resampling phân tầng được cung cấp cho phân tích điều kiện trên cohort và trọng số đã khai báo; cluster resampling là kiểm tra độ nhạy theo group. Đây không phải một bộ ước lượng phương sai survey tổng quát cho mọi thiết kế vừa phân tầng vừa phụ thuộc nhóm. Khi cần suy luận population phức tạp, phải khóa thiết kế và bổ sung công cụ phù hợp trước kết luận.

Nếu resample có metric không xác định, output đếm các lượt đó; CI chỉ từ lượt xác định được gắn nhãn diagnostic và không thay CI danh nghĩa. Candidate model có MCC không xác định không tự được thay bằng 0 để chọn winner. Exact McNemar dùng số đếm đúng/sai thô, không đưa trọng số vào phép nhị thức.

13 Kế hoạch thời gian và nguồn lực

13.1 Các mốc công việc theo phụ thuộc

Thời gian mục tiêu	Trọng tâm	Sản phẩm và điều kiện chuyển bước
Ngày 1 đến 3	Canonical joins, clean mask, repair/harm, prompt audit	Bảng offline và danh sách confound phải xử lý

Thời gian mục tiêu	Trọng tâm	Sản phẩm và điều kiện chuyển bước
Ngày 4 đến 6	Gate chung, meta controls, cohort ngoài, capability pilot	Protocol, cohort và score contract được khóa
Tuần 2	Expert development và xác nhận, baseline ba seed	Prediction cùng deployment và log chi phí
Tuần 3	Routing, CI, repeat và packet lỗi	Bảng chính, budget curves và bằng chứng theo code
Tuần 4	Viết bài và gói tái lập	Bản bài lõi, quyết định scope và track
Tuần 5 đến 8 nếu cần	Hạ tầng tự host, score và probe mở rộng	Bằng chứng bổ sung có kiểm soát, không tìm winner hậu nghiệm

Đây là dự toán khi code/data/artifact và endpoint sẵn sàng. Không tự chốt một ngày đăng ký trước hoặc nộp bài khi chưa xác minh tài nguyên/CFP. Thời gian hạ tầng, chấm tay và xử lý nhãn cần theo dõi riêng trong PROGRESS.md; có thể viết từ khi bảng lõi đã ổn, không đợi mọi probe mở rộng.

13.2 Dự toán truy vấn và tài nguyên

Đường lõi chuyển dữ liệu từ ReVeal có thể cần khoảng 2.196 response E1 development, 4.000 response E0/E1 cho 2.000 target ngoài và khoảng 300 pilot/repeat tăng thêm, tức khoảng 6.500 response cho expert chính trước retry. Đây là lượng việc minh họa, không báo giá. Nếu lặp đầy đủ backbone hai cho cùng cặp meta và E0/E1, có thể thêm khoảng 6.200 response trước repeat/retry; chỉ lặp mô tả trên một subset không phải xác nhận meta đầy đủ.

Nếu có cache exact contract, chi phí vật lý giảm; query theo thuật toán vẫn báo riêng. Nếu chuyển sang self-host, đo throughput/VRAM và đơn giá tại ngày pilot. Chi phí học và chuẩn bị, expert development, full reference, inference triển khai và archive đều cần có dòng riêng. Không khẳng định API bằng 0 nghĩa nghiên cứu không có chi phí hoặc việc tự host luôn rẻ nhất.

13.3 Mức sản phẩm và điểm dừng chi phí

Vài ngày offline tạo báo cáo nền và bộ kiểm đúng luồng, chưa tự đủ cho full paper. Gói bài lõi thêm holdout ngoài, matched controls, repeat/seed và cost có truy vết. Gói sáu đến tám tuần có thể bổ sung scoring/self-host và probe cơ chế khi cần. Phần mở rộng tạp chí được quyết định theo câu hỏi mới mà kết quả bài đầu làm rõ; ba đến bốn tháng chỉ là mốc lập kế hoạch có điều kiện, không phải bảo đảm công bố Q1.

14 Điều kiện dừng và cách xử lý kết quả âm

Nếu lợi thế classifier biến mất sau overlap/label audit, dừng khẳng định nền và sửa dữ liệu trước. Nếu graph/demo không thực sự đến model, ưu tiên sửa contract rồi đo lại cặp cần thiết; không kết luận chuyên môn từ một đầu vào bị mất phần chính.

Nếu expert có nhiều ca sửa đúng nhưng P_post không vượt P_pre, có thể tín hiệu chưa học được ổn định, meta quá yếu, hoặc các ca đúng chỉ là biến động; cần xem CI và repeat. Không mặc định tăng độ sâu stacker sẽ giải quyết. Nếu P_post có lợi nhưng router không hơn confidence/random, đóng góp nằm ở khoảng cách trước/sau truy vấn; không báo “tiết kiệm chi phí” bằng selector cần 100% query.

Nếu cả P_post và router không có lợi trên holdout ngoài, chốt kết quả âm có phạm vi và phân tích nguyên nhân. Không mở thêm model hoặc threshold sau khi xem test chỉ để tìm một winner rồi gọi nó là xác

nhận. Một kết quả âm chặt có thể có giá trị, nhưng vẫn cần câu hỏi mới và bằng chứng đủ mạnh để vượt mức ghi nhận "không cải tiến được".

Nếu các nguồn gần nhất đã trả lời phần lớn RQ và nhóm chỉ lặp lại trên hai benchmark cũ, điều chỉnh mục tiêu thành nghiên cứu tái lập/negative-result hoặc bổ sung một câu hỏi chuyển miền cụ thể. Không coi yêu cầu tìm tính mới là lý do để mở đồng thời cả H1/H2/H3.

15 Các khả năng suy luận chưa được chấp nhận

Phần này thống kê **đúng 26 nhóm luận điểm nền tảng**, từ D01 đến D26. Một luận điểm được sử dụng ở nhiều đoạn chỉ được tính một lần; các ví dụ và hệ quả của cùng một phép suy luận được trình bày trong cùng nhóm. Có nhóm chứa nhận định sai, có nhóm đúng một phần nhưng đi xa hơn bằng chứng, và có nhóm là một khả năng hợp lý chưa đủ dữ liệu để chấp nhận. Vì vậy, "chưa được chấp nhận" không đồng nghĩa mọi thành phần của nhóm đều sai. Với mỗi nhóm, cần giữ phần có căn cứ và thay kết luận quá mạnh bằng phép kiểm tra cụ thể.

D01 Từ MCC nhỏ đến kết luận không chuyển được

Một khả năng suy luận là MCC khoảng 0,02–0,10 chứng minh GRACE hoạt động ngẫu nhiên và không chuyển được sang LLM open weight. Các giá trị nhỏ cho thấy khả năng phân loại cân bằng còn yếu ở những cấu hình đã đo, nhưng không tự xác lập phân phối ngẫu nhiên của predictor. Kết luận về tính chuyển được còn phụ thuộc backbone, phiên bản, prompt, ngân sách, dữ liệu và cách tái cài đặt.

Cách viết phù hợp là các run hiện tại chưa cho thấy hiệu quả mạnh của nhánh GRACE trong điều kiện đã khai báo. Muốn kiểm tra khả năng chuyển backbone, cần giữ các thành phần còn lại cố định hoặc mô tả rõ sai khác, đo khoảng tin cậy và xác nhận trên phạm vi mới. Không mở rộng một kết quả có điều kiện thành nhận định về mọi LLM open weight.

D02 Gán kết quả winner cho hồi quy logistic thuần

Có thể cho rằng MCC khoảng 0,30 trên Devign và 0,37 trên ReVeal thuộc một hồi quy logistic chỉ nhận embedding CodeT5. Hồ sơ H3-B hiện mô tả việc lựa chọn giữa các classifier semantic, lexical, MLP và fusion; tên policy gate_tuned chưa đủ nhận diện kiến trúc base winner. Vì thế, không được gán kết quả này cho semantic LR nếu chưa đối chiếu manifest và model artifact tương ứng.

Trong bảng kết quả lịch sử, dùng tên "classifier hoặc fusion đã được chọn" và ghi chính xác base winner sau kiểm tra. Hồi quy logistic trên embedding vẫn là baseline nên chạy riêng, nhưng số của baseline mới phải lấy từ prediction của chính nó. Việc sử dụng cùng encoder không làm hai classifier trở thành cùng một phương pháp.

D03 Đồng nhất chưa vượt có ý nghĩa với không vượt về điểm

Khả năng suy luận rằng không nhánh phối hợp nào vượt gate_tuned chưa chính xác về mặt số học. Trên Devign, stacking LR có MCC 0,3013, cao hơn gate_tuned 0,2961 theo điểm ước lượng. Tuy nhiên, CI của chênh lệch được báo cáo chứa 0, nên chưa có bằng chứng rõ về lợi ích. Đây là hai nhận định khác nhau.

Cần ghi cả effect size và độ bất định, đồng thời giữ winner đã chọn trên development. Không thay winner bằng nhánh có điểm test tốt hơn. Tương tự, control gate ở ngưỡng 0,5 có thể có điểm cao hơn gate_tuned nhưng không được chọn lại hậu nghiệm. Kết luận phù hợp là các nhánh phối hợp chưa xác nhận được lợi ích tăng thêm so với classifier nền trong lượt đánh giá hiện tại.

D04 Từ hybrid chưa thắng đến không có tín hiệu bổ sung

Việc một router hoặc stacker chưa thắng classifier không chứng minh expert hoàn toàn không có thông tin bổ sung. Theo số đếm đối soát cần xác nhận lại từ từng ID, expert vẫn đúng ở 303 trường hợp classifier sai trên Devign và 218 trường hợp trên ReVeal. Vấn đề có thể nằm ở việc nhận diện các trường hợp hữu ích, độ ổn định của expert hoặc lớp mô hình phối hợp được sử dụng.

Nghiên cứu phải tách bổ sung có sẵn, bổ sung khai thác sau truy vấn và bổ sung khai thác trước truy vấn. Cặp meta-model có và không có feature expert kiểm tra giá trị dự đoán tăng thêm trong lớp mô hình đã chọn; routing cùng ngân sách kiểm khả năng chọn query hữu ích. Không kết luận không có tín hiệu chỉ vì một cơ chế phối hợp chưa khai thác được nó.

D05 Diễn giải độ nhạy nhãn thành tỷ lệ sao chép

Đảo nhãn demonstration rồi thấy prediction thay đổi chứng minh mô hình nhạy với nhãn được trình bày, chưa chứng minh mô hình chỉ sao chép nhãn và không sử dụng code. Một demonstration đúng nhãn nhưng khác nhãn target là tình huống ICL bình thường; giữ nguyên code và cố ý gán sai nhãn tạo thông tin mâu thuẫn. Không được coi hai can thiệp này là tương đương. Các nghiên cứu ICL liên quan cần được đối chiếu theo đúng thiết kế, không dùng như bảo đảm cho một cơ chế ở hệ thống đang xét [T20–T21].

Hồ sơ gọi 31% là Net Label Sensitivity và 4% là Natural Variance. Trước khi tính lại, phải xác minh 31% đã trừ biến động hay chưa. Báo raw flip rate, repeat disagreement, phần chênh và các ca đúng thành sai, sai thành đúng. Cặp tương phản giảm độ nhạy cũng chưa tự chứng minh cân bằng nhãn là nguyên nhân duy nhất.

D06 Từ Neither đến kết luận kho không có mẫu hữu ích

Nhận định 30 trong 50 target Devign thuộc nhóm Neither không chứng minh 60% target không có demonstration phù hợp trong toàn bộ kho. Phiếu chấm chỉ phản ánh những selections được đưa vào packet, rubric và ngữ cảnh có sẵn. Một retriever có thể bỏ lỡ mẫu tốt; code fragment hoặc graph có thể thiếu thông tin; người chấm cũng có thể chưa xác định được quan hệ bảo mật.

Cách kết luận phù hợp là cả hai selections được đánh giá chưa cung cấp bằng chứng liên quan theo rubric ở những trường hợp đó. Muốn quy nguyên nhân cho bank, cần kiểm tập ứng viên rộng hơn hoặc một tập oracle được thẩm định độc lập. Giữ nhóm không đủ ngữ cảnh riêng, báo đồng thuận trước hòa giải và không suy tỷ lệ của mẫu chấm có phân tầng thành tỷ lệ toàn dataset.

D07 Từ kết quả H1 và H2 đến kết luận không dùng nội dung

Một khả năng là H1 chưa cải thiện chứng minh truy hồi tốt hơn không có ý nghĩa, còn H2 chỉ đáng nghiên cứu sau khi chứng minh LLM đọc graph. Lập luận này đi trước bằng chứng. H1 có các trường hợp demo bị cắt rỗng và prompt giống nhau nhưng response khác; H2 chưa có đầy đủ prefix control cùng điều kiện. Chưa thể quy kết quả cho việc model không sử dụng nội dung.

Cần audit đầu vào thực nhận rồi thực hiện các phép so đúng thành phần. Một representation mới có thể giúp chính vì sửa được cách biểu diễn khiến model trước đó khó khai thác graph. Vì vậy, probe graph là công cụ chẩn đoán và H2 là một can thiệp có thể kiểm tra; không biến một phép thử nhị phân đọc hoặc không đọc thành điều kiện bắt buộc trước mọi cải tiến.

D08 Xem kết quả tiết kiệm nhỏ là không có giá trị thực nghiệm

Có thể suy rằng ràng buộc bỏ sót 1% làm vùng tự quyết của H3-A rỗng và kết quả không có giá trị. Trên Devign, cascade đã skip sáu mẫu, nên vùng đó không rỗng; trên ReVeal, không có mẫu được skip. Mức

giảm query khoảng 0,22% ở Devign quá nhỏ để làm bằng chứng tiết kiệm đáng kể trong cấu hình hiện tại, nhưng vẫn là một kết quả về đánh đổi rủi ro và chi phí.

Nên giữ H3-A như đối chứng hoặc kết quả âm có phạm vi, báo rõ ngưỡng, cận validation và query ratio theo thuật toán. Không coi số request mới sau cache là mức tiết kiệm. Chưa nên đầu tư mở rộng H3-A làm hướng chính khi chưa có mục tiêu chất lượng và chi phí đủ hấp dẫn, nhưng không loại bỏ giá trị khoa học của một ràng buộc cho thấy khó đạt tiết kiệm.

D09 Tuyên bố đầu tiên khi chưa đối chiếu nghiên cứu gần

Các mệnh đề chưa có tái lập độc lập của GRACE, chưa ai kiểm tác động graph hoặc demonstration, và chưa có baseline nhẹ để định vị LLM đều cần kiểm tra lại. Đã có các nghiên cứu rất gần về tái lập nhiều hệ RAG gồm GRACE, code metrics, chất lượng benchmark, ensemble và ảnh hưởng của protocol. Một ý tưởng không xuất hiện trong abstract không có nghĩa toàn văn chưa nghiên cứu nó.

Đóng góp nên được xác lập bằng bảng đối chiếu câu hỏi, dữ liệu, baseline, controls và phạm vi xác nhận của các công trình gần nhất. Nếu phần lớn phép thử đã có, giá trị mới phải nằm ở câu hỏi khác, chẳng hạn marginal utility so cùng classifier và ngưỡng, khả năng khai thác dưới chuyển miền hoặc chi phí thực. Không giữ tuyên bố đầu tiên chỉ bằng việc đổi tên khung thí nghiệm.

D10 Dùng khoảng tin cậy chứa không để xác nhận không tác dụng

Một số giả thuyết có thể được diễn đạt rằng demonstration không mang tín hiệu, graph không có tác dụng hoặc stacking không bổ sung giá trị nếu CI của chênh lệch chứa 0. Đây là cách diễn giải không đúng. Khoảng chứa 0 có thể đồng thời bao gồm một mức lợi ích đáng quan tâm; khi đó dữ liệu chỉ chưa đủ để phân biệt các khả năng.

Muốn loại được lợi ích ít nhất bằng một biên thực tiễn, phải khóa biên và cách dùng cận một phía trước phép xác nhận. Muốn khẳng định tương đương cần biên hai phía và phương pháp phù hợp. Mỗi bảng phải ghi effect size, CI và câu hỏi thống kê được trả lời. Không đổi tiêu chí sau khi xem kết quả để biến một phép thử chưa đủ lực thành bằng chứng không tác dụng.

D11 So F1 bài gốc với đối chứng hằng của cohort khác

Khả năng suy luận rằng F1 65,11% của bài gốc chỉ hơn always-vulnerable khoảng ba điểm nên gần tầm thường chưa được chấp nhận. Mốc hằng phải tính trên đúng cohort và phân bố nhãn của phép đánh giá gốc; số suy từ cohort tái cài đặt chưa đủ thay thế. Ngay khi cùng cohort, chênh F1 nhỏ với predictor hằng cũng không tự cho biết MCC, số báo động giả hay sự đánh đổi precision và recall.

Cần tách đối chiếu công bố lịch sử khỏi so sánh thành phần được chạy trong cùng điều kiện. Không dùng một ngưỡng MCC tùy chọn hoặc việc F1 thấp hơn hằng để đại diện cho toàn bộ câu hỏi tái lập. Khi thiếu prediction hoặc thông tin split gốc, giới hạn kết luận đúng phạm vi thay vì suy rằng hiệu quả công bố đã bị bác bỏ.

D12 Đồng nhất bằng AUC với chỉ khác điểm hoạt động

Điểm số liên tục giúp đánh giá ranking qua nhiều ngưỡng, nhưng hai arm có cùng AUROC không nhất thiết có cùng thứ hạng từng mẫu hoặc cùng đường ROC. Hai đường có thể cắt nhau và có diện tích bằng nhau; CI của chênh lệch chứa 0 cũng chưa chứng minh diện tích bằng nhau. Vì thế, không thể từ AUC tương tự và MCC khác nhau kết luận toàn bộ khác biệt chỉ là đổi điểm hoạt động.

Cần đọc AUROC cùng average precision, đường cong, operating metrics và độ bất định trên cùng cohort. Một ngưỡng như AUROC không quá 0,62 không tự định nghĩa predictor ngẫu nhiên; mốc ranking không

thông tin là 0,5 và phép suy luận còn phụ thuộc CI. Không dùng đánh giá không ngưỡng như công cụ giải quyết dứt điểm mọi câu hỏi về cơ chế.

D13 Dùng graph sai ghép để xác định model đọc graph

Thay CPG của target bằng CPG từ một hàm khác cùng kích thước là một probe hữu ích, nhưng phải gọi đúng là graph sai ghép. Graph thay thế vẫn có nội dung và có thể tạo bất nhất với code; số node gần nhau không bảo đảm token, edge type, identifier hoặc phân bố thuộc tính giống nhau. Do đó, phép can thiệp không chỉ đơn giản loại một khối thông tin và giữ nguyên mọi yếu tố khác.

Nếu verdict đổi, có thể kết luận model nhạy với can thiệp đó, chưa đủ kết luận model sử dụng graph đúng ngữ nghĩa bảo mật. Nếu không đổi, có thể do code đã đủ, probe yếu hoặc nhiễu lớn. Cần giữ donor trong train, log ID và budget, bổ sung control khi cần, và tránh dùng một probe để quyết định tuyệt đối có hay không có khả năng khai thác graph.

D14 Coi mọi probe demonstration đều cô lập nội dung

Demo ngẫu nhiên không khớp nhãn và độ dài có thể làm thay đổi class prior được trình bày, context và nội dung cùng lúc. Demo bỏ nhãn so với code-only cũng không đo riêng demonstration nếu một bên còn có graph. Tương tự, label-only bỏ code demo nên thay cả chiều dài và cấu trúc, không phải đối chứng giữ nguyên hình thức hoàn hảo.

Để đo nội dung truy hồi, ưu tiên demo ngẫu nhiên khớp nhãn và khoảng độ dài, bên cạnh random policy không ràng buộc được đặt tên riêng. Để đo demo không nhãn khi đã có graph, so với cấu hình graph không demo. Muốn đo riêng nhãn, giữ đúng demo và các phần còn lại. Mỗi probe phải nêu phần thực sự thay đổi và giới hạn suy luận; không chỉ dựa vào tên arm để khẳng định đã cô lập cơ chế.

D15 Dùng baseline tốt để loại trừ vấn đề dữ liệu

Classifier có MCC dương cho thấy có tín hiệu dự đoán khai thác được trong representation và phân bố đang xét, nhưng không chứng minh dữ liệu không có vấn đề. Tín hiệu có thể chứa shortcut, đặc trưng project hoặc overlap; nhãn nhiễu và thiếu context có thể tồn tại đồng thời với hiệu năng classifier tương đối tốt. Vì thế, không được quy toàn bộ khoảng cách với LLM cho cách dùng LLM.

Cần giữ overlap audit, phân tích khác project hoặc thời gian và kiểm tra nhãn dựa trên evidence. Baseline nhẹ là mốc quan trọng để đánh giá giá trị tăng thêm và để phát hiện kết quả bất thường, không phải chứng thư rằng dataset phản ánh đầy đủ trạng thái bảo mật. Mọi kết luận nguyên nhân phải được hỗ trợ bằng can thiệp hoặc phân tích riêng.

D16 Đồng nhất score verbalizer với xác suất lỗ hổng

Score chuẩn hóa từ xác suất hai candidate 0 và 1 là một đại lượng điều kiện theo verbalizer và protocol đã chọn. Nó chưa tự là xác suất hàm có lỗ hổng được hiệu chuẩn. Candidate có nhiều token, khoảng trắng hoặc token khác ngoài hai chữ số còn có thể làm candidate argmax khác verdict được sinh tự do. Vì vậy, s lớn hơn hoặc bằng 0,5 chưa chắc chính là câu trả lời decoded của model [T22].

Cần lưu riêng raw verdict, candidate score, candidate argmax, trạng thái hợp lệ và nhãn sau ngưỡng validation. Tokenizer, prefix và cách tính candidate likelihood phải được khóa. Nếu cần xác suất triển khai, hiệu chuẩn trên dữ liệu phù hợp và kiểm lại dưới shift; không gán ý nghĩa xác suất bảo mật chỉ vì score nằm trong khoảng 0 đến 1.

D17 Xem tự host là điều kiện bắt buộc và bảo đảm tất định

Không có suy luận tổng quát rằng muốn logprob thì bắt buộc tự host. Khả năng lấy score phụ thuộc backend và API cụ thể; một API đủ khả năng có thể đáp ứng thí nghiệm. Tự host giúp kiểm soát phiên bản

và logging, nhưng seed cố định không bảo đảm mọi kết quả giống nhau trên mọi GPU, kernel, batch và phiên bản phục vụ. Nó cũng không loại hoàn toàn lỗi hạ tầng hoặc response [T17].

Lựa chọn hạ tầng phải dựa trên capability và smoke test, đồng thời pin model revision, tokenizer, quantization và môi trường [T19]. Báo repeat disagreement như một phép đo, không như một lỗi mặc định đã được xóa nhờ vLLM. Việc chuyển backend tạo session mới và cần kiểm tác động lên expert development dùng học meta-model.

D18 Gộp các snapshot thành một bằng chứng đồng nhất

Các artifact lịch sử không tự tạo thành ba backbone có đủ cùng thang code, graph và demo. Model, provider, ngân sách và builder đã thay đổi; chỉ một số phép ghép cặp có đủ bằng chứng cùng điều kiện. Tương tự, sáu target cùng prompt khác prediction ở một snapshot không thay thế thống kê 77 trong 2.179 ở bảng cập nhật của một phạm vi khác. Mỗi số cần gắn run, cohort và thời điểm.

Các số duplicate 108 và 52 của một audit cũng không được thay cho overlap train hoặc development của H3-B là năm và 63 test record. Devign development đủ điều kiện 2.724 khác official validation gốc 2.732. Cần bảng provenance và quy tắc chọn snapshot; không dùng một số đúng ở run này để điền ô còn thiếu của run khác.

D19 Coi đăng ký trước là cách làm mới test đã xem

Đăng ký protocol trước một lượt chạy mới có ích, nhưng không biến test đã được dùng phát triển thành holdout độc lập. Một nhãn chưa mở lại trong phiên làm việc mới vẫn có thể đã ảnh hưởng các quyết định trước đó. Khử exact duplicate hoặc khóa mask chỉ cải thiện một khía cạnh độc lập dữ liệu, không xóa exposure của quá trình nghiên cứu.

Cần gọi phân tích trên test cũ là thăm dò, tái lập hoặc độ nhạy, và giới hạn kết luận tương ứng. Phép xác nhận khái quát cần cohort ngoài hoặc split project hay thời gian chưa dùng chọn phương án. Nếu chưa có điều kiện này, nêu giới hạn và chọn phạm vi công bố phù hợp; không để mọi xác nhận mới sang bước mở rộng nhưng vẫn mô tả bài đầu như đã có kiểm chứng độc lập.

D20 Suy độ rộng khoảng tin cậy chỉ từ cỡ mẫu

Cỡ mẫu 100 không tự xác định sai số MCC là khoảng cộng trừ 0,2. Độ bất định còn phụ thuộc tỷ lệ lớp, confusion matrix, cấu trúc bất đồng của hai phương pháp và quan hệ phụ thuộc giữa các mẫu. Với chênh lệch metric, hai predictor có thể đạt điểm gần nhau nhưng đúng sai trên các ID rất khác nhau.

Vì vậy, cỡ mẫu nhỏ là lý do phải tính uncertainty và tránh kết luận mạnh, không phải căn cứ để đưa một khoảng chung. Dùng prediction development để ước lượng CI hoặc mô phỏng các cấu trúc lỗi cho kế hoạch cỡ mẫu. Báo số positive và group, xử lý replicate không xác định, và không thay phép tính cụ thể bằng một con số sai số có vẻ hợp lý.

D21 Tạo score từ logprob thiếu bằng giá trị giả

Nếu chỉ có top-k token và thiếu candidate 0 hoặc 1, điền logprob bằng một hằng số như −30 có thể tạo score tự tin giả. Điều kiện hợp lệ kiểu có ít nhất một candidate không bảo đảm tính được tỷ số hai candidate. Việc strip khoảng trắng rồi dùng dictionary cũng có thể ghi đè các token variant thay vì xử lý xác suất theo định nghĩa verbalizer.

Nên dùng candidate scoring đầy đủ với tokenization được kiểm, hoặc báo score missing và giữ decoded verdict theo quy tắc riêng. Không tự thay protocol bằng constrained decoding rồi xem kết quả hoàn toàn tương đương. Structured output hoặc lựa chọn ép buộc phụ thuộc API và phiên bản, phải được smoke

test và khóa trước xác nhận [T18, T22]. Mẫu thiếu score vẫn phải được giữ trong coverage và phân tích lỗi.

D22 Xem chuẩn hóa và tìm clone gần đúng là kiểm toán đầy đủ

Hash sau bỏ comment hoặc gộp whitespace là normalized hash, khác raw exact hash. Một cách xử lý C/C++ bằng regex có thể đụng tới string literal hoặc cấu trúc có ý nghĩa; Jaccard token với ngưỡng cố định và tìm trong top-50 láng giềng cũng chỉ là một detector clone gần đúng. Không thể từ đó tuyên bố đã loại hết rò rỉ.

Cần giữ raw và normalized hash riêng, phiên bản chuẩn hóa, exclusion log và đánh giá một số ca gần ngưỡng. Nếu thay train hoặc retrieval pool, refit các thành phần bị ảnh hưởng và tạo cache identity mới. Replay metric trên mask cũ và chạy lại hệ thống với bank đã làm sạch là hai thí nghiệm khác nhau; không sử dụng response cũ như thể demo và prompt chưa thay đổi.

D23 Coi mã minh họa là công cụ tái lập hoàn chỉnh

Một số lựa chọn mã tiện lợi có thể phá tính tái lập: hash mặc định của chuỗi thay đổi giữa process; cache chỉ theo prompt hash bỏ sót model, tokenizer và cấu hình suy luận; bootstrap trả trung bình chênh lệch lấy mẫu lại thay cho effect size quan sát. Những vấn đề này cần sửa trước khi mã được dùng tạo bảng chính.

Dùng stable digest từ dữ liệu đã chuẩn hóa cách serialize, cache theo đủ request identity, và báo observed delta tách khỏi phân bố bootstrap. Kiểm replicate chỉ có một lớp hoặc metric có mẫu số bằng 0. Kiểm không rò nhãn target bằng cách thay hoặc bỏ field đó rồi xác nhận selections và prompt không đổi; không quét ký tự 0 hoặc 1 trong code để kết luận có rò nhãn.

D24 Chốt thời gian và chi phí khi chưa đo throughput

Tuyên bố phần lớn dữ liệu đã sẵn sàng, khoảng 120.000 prompt là hướng rẻ nhất, hoặc toàn chiến dịch chỉ cần một mức giờ GPU và tiền thuê cố định đều cần inventory và benchmark cụ thể. Phép nhân 120.000 prompt với trung bình 2.500 token cho khoảng 300 triệu input token là một ước lượng khối lượng; thời gian còn phụ thuộc model, quantization, độ dài, batch, KV cache, concurrency và hạ tầng.

Nên đo pilot để dự toán, tách tải model, tạo graph, inference, retry và chấm thủ công. Ma trận rộng là bước mở rộng, không bắt buộc của bài đầu. Tiến hành theo các tầng có điểm dừng; chỉ tăng số arm khi chúng cần cho kết luận. Thời gian ba, bốn hoặc tám tuần là kế hoạch có điều kiện, không phải bằng chứng phương án nào ít tốn nguồn lực hơn.

D25 Bảo đảm công bố bằng nhãn track hoặc tỷ lệ nội dung mới

Tái lập và kết quả âm có thể có giá trị, nhưng không tự bảo đảm acceptance. Một bài vẫn cần câu hỏi đáng quan tâm, thiết kế đúng, bằng chứng và khác biệt với prior art. Nhãn hội thảo, workshop, research track, quartile và chỉ mục không được sử dụng thay cho đánh giá mức đóng góp; thông tin các kỳ và category phải được kiểm tại thời điểm nộp.

Tương tự, không áp một quy tắc ít nhất 30% nội dung mới cho mọi journal extension, hoặc hứa đạt Q1 sau một số tháng. Cần tuân thủ chính sách venue cụ thể, giải thích phần kế thừa và bổ sung câu hỏi hay bằng chứng có thực chất. Các tên tạp chí là hướng đích chuyên môn, không phải chứng nhận Q1 hiện hành ở mọi năm và ngành.

D26 Xem cặp vá lỗi là demonstration tốt nhất và phép bác bỏ ICL

Cặp hàm lỗi và bản vá thật có provenance tốt và là hướng kiểm tra quan trọng, nhưng chưa chắc là demonstration tốt nhất cho mọi target. Patch có thể sửa nhiều vấn đề, phụ thuộc context ngoài hàm hoặc

không tương đồng cơ chế với target. Một kết quả không cải thiện với cách chọn cặp đó không bác dứt điểm mọi hình thức ICL.

Nên dùng patch pairs từ train với nguồn gốc và grouping rõ, tránh để target hoặc bản vá liên quan lọt vào demo bank. Đánh giá riêng tính đúng của cặp, độ phù hợp với target và lợi ích downstream; khi kiểm pairwise prediction phải giữ đủ hai thành viên. Kết luận chỉ áp dụng cấu hình đã đo. Nếu có lợi ích, xác nhận lại trước khi gọi đó là một cơ chế tổng quát; nếu không, giữ kết quả âm đúng phạm vi.

16 Định vị bài hội thảo và lộ trình mở rộng tạp chí

16.1 Gói đóng góp phù hợp cho bài đầu

Tên bài tiếng Anh có thể là **When Does a Graph Augmented LLM Add Value to a Lightweight Vulnerability Classifier**. Tên này nêu câu hỏi cần trả lời, không ngầm khẳng định hybrid đã thắng. Nếu kết quả chính là giới hạn khai thác, title và abstract phải phản ánh điều đó.

Bài có thể nhắm các họ hội thảo phù hợp với nghiên cứu thực nghiệm và phân tích phần mềm như ESEM, SANER hoặc QRS, theo track và kỳ còn phù hợp [T15–T16]. Đây là định vị chuyên môn, không phải xác nhận đang mở nhận bài. ESEM có hướng nghiên cứu thực nghiệm/tái lập; SANER có các đường research và reproducibility/negative results. Workshop, short paper và main research track có yêu cầu bằng chứng và mức công nhận khác nhau; không gọi một workshop đi kèm hội thảo lớn là bài main conference.

Để nhắm full paper, không chỉ gửi bảng classifier vượt GRACE. Cần tối thiểu câu hỏi khác với prior art gần nhất, matched controls, bằng chứng mới ngoài test đã dùng, phân tích lỗi và chi phí có truy vết. Hai backbone và một can thiệp context có kiểm soát sẽ làm lập luận vững hơn; không có ngưỡng số model/dataset nào tự động bảo đảm acceptance.

16.2 Dàn ý bài nghiên cứu khi đã có kết quả

Introduction trình bày câu hỏi giá trị bổ sung và ba đóng góp thực tế đã đạt. Background giải thích graph/ICL, classifier và defer. Related Work xác định phần đã có và khác biệt được bảo vệ. Study Design mô tả cohort, học/chọn cấu hình, expert contract, metric, chi phí và primary contrasts. Results lần lượt trả lời RQ1–RQ4. Discussion giải thích điều kiện dùng expert và các kết quả âm. Threats to Validity trình bày leakage, label noise, serving shift, chuyển miền và giới hạn mức hàm. Conclusion chỉ nhắc các kết quả đã đo.

Không đặt số dự kiến vào abstract của bài nộp. Giữ các bảng thăm dò lịch sử trong motivating evidence hoặc appendix, tách khỏi bảng xác nhận. Bài đầu có thể dành khoảng 1–1,5 trang cho động cơ, 2–3 trang cho thiết kế, phần còn lại cho kết quả/giải thích tùy giới hạn venue; không chép nguyên đề cương dài vào bài.

16.3 Hướng mở rộng có chiều sâu cho tạp chí

Đường mở rộng đề nghị là **khả năng dự đoán giá trị expert dưới thay đổi project/thời gian và chất lượng evidence**. Giai đoạn này kế thừa câu hỏi bài đầu, nhưng bổ sung một cơ chế và kiểm chứng mới. Chỉ chọn H2 khi case study cho thấy expert thất bại vì thiếu hoặc đứt liên kết thao tác–guard trong context.

Khi đó so prefix, slice và H2 giữ cụm evidence trong cùng graph source, cùng target/demo, cùng budget/renderer và cùng expert. Đối chứng bỏ guard chỉ thay phần guard, không lấp bằng nội dung khác. Đánh giá evidence fidelity, polarity/unknown và counterevidence bởi nguồn độc lập; không lấy score heuristic của chính H2 làm đáp án. Nếu một thao tác được guard nhưng code vẫn lỗi vì alias/overflow/nhánh khác, phải ghi đúng giới hạn đó.

Tiếp theo kiểm tra router có nhận diện được **khả năng expert được hưởng lợi từ evidence** từ feature pre-query hay không. Feature có thể mô tả graph availability, guard coverage quan sát được và code length, nhưng không dùng nhãn lỗ hổng hoặc response tương lai của target. So với confidence, benefit lỗi 0/1, benefit cân bằng lớp và gate-only, ở cùng budget. Không tuyên bố quan hệ nhân quả bảo mật chỉ vì thêm một feature graph làm score cao.

Trục mở rộng	Bài đầu	Bản tạp chí cần thêm
Câu hỏi	Giá trị expert vượt classifier	Khi nào giá trị đó khái quát và có thể dự đoán dưới shift
Dữ liệu	Hai cohort cũ và một cohort ngoài	Nhiều project/time windows, dữ liệu target-domain và paired fixes
Model	Một expert chính, ưu tiên lặp expert thứ hai	Ít nhất vài expert khác họ/quy mô và serving ổn định
Cơ chế	Matched controls, repair/harm	Can thiệp evidence và router có đối chứng tách thành phần
Ứng dụng	Hàm theo benchmark	Báo động/TP, false positives, provenance patch và chi phí thực
Độ tin cậy	Seed, repeat, CI	Phân tích drift, calibration/risk và xác nhận ngoài nhóm phát triển

Empirical Software Engineering hoặc Journal of Systems and Software là các hướng đích chuyên môn tự nhiên; TSE/TOSEM đòi hỏi độ sâu và khả năng khái quát cao hơn. Khi chọn tạp chí phải kiểm tra SCIE và JCR quartile theo đúng năm/ngành nếu yêu cầu là ISI Q1; không đồng nhất nhãn Scopus/SJR Q1 với JCR Q1. Không mặc định một tên tạp chí luôn Q1 ở mọi category và mọi năm.

Bản mở rộng phải có câu hỏi, dữ liệu hoặc phương pháp mới có thực chất, trích dẫn và giải thích phần kế thừa bài hội thảo, tuân thủ chính sách venue đích. Không dùng một tỷ lệ phần trăm nội dung mới chung cho mọi tạp chí. Thêm vài benchmark hoặc kéo dài Related Work chưa đủ tạo thành một bài tạp chí mạnh.

16.4 Các mở rộng được tiếp thu có điều kiện

Big-Vul có thể giúp đối chiếu thêm phạm vi bài gốc; PrimeVul hỗ trợ chuyển dữ liệu, metadata và đánh giá vulnerable/fixed pairs. Không dời mọi xác nhận mới sang bản tạp chí nếu bài đầu muốn kết luận khái quát ngoài test đã xem. Cặp demo lỗi–đã vá phải lấy từ train, có provenance, không phải patch của chính target hoặc near-clone làm lộ nhãn. Đánh giá cả pair correctness và trường hợp cùng gán 1/cùng gán 0; một kết quả âm không bác toàn bộ ICL.

Có thể thêm trục kích thước cùng họ model, model mạnh khác họ và baseline fine-tune CodeBERT/UniXcoder khi nguồn lực cho phép. Giữ quantization và serving contract tương thích hoặc khai báo như biến thí nghiệm. Không gọi một cặp vá thật là demonstration tốt nhất có thể. Kiểm toán nhãn trong FP/FN là bằng chứng có điều kiện, không tự cho trần hiệu năng do noise.

Theo dõi CFP, category và chính sách journal extension ở đúng kỳ/năm. Không mặc định CORE/ranking, hạn nộp giữa năm hoặc quy tắc ít nhất 30 phần trăm nội dung mới áp dụng cho mọi venue. Registered report có quy trình phản biện protocol riêng; một commit trước lượt chạy mới không tự là registered report được venue chấp nhận.

17 Rủi ro đối với độ tin cậy của nghiên cứu

Tính đúng bên trong. Sai parser, nhãn mapping, cache identity, feature sau expert lọt vào router, preprocessing ngoài fold hoặc fitter bỏ mất demo đều có thể tạo kết luận sai. Biện pháp là canonical joins, hash, kiểm fold, audit actual prompt và matched controls.

Tính đo lường. Nhãn benchmark không đồng nghĩa đã chứng minh trạng thái bảo mật của cả hệ thống. MCC/F1 và accuracy đo những khía cạnh khác nhau; repair/harm theo lỗi 0/1 không tối ưu MCC. Báo đủ confusion và nguồn nhãn, kiểm một số ca bằng patch/metadata.

Tính khái quát. Hai dataset cũ, C/C++ ở mức hàm, một provider hoặc một model không đại diện mọi ngôn ngữ/công cụ. Cohort PrimeVul transfer kiểm một dạng thay đổi dữ liệu; chưa thay thế đánh giá whole-project. Các project có chung nguồn phát triển có thể vẫn phụ thuộc.

Tính kết luận thống kê. Nhiều threshold/candidate và test exposure gây lạc quan; ít positive/cluster làm CI bất ổn. Khóa contrasts và cohort mới, báo mọi seed và phép đo, hiệu chỉnh nhóm kiểm định xác nhận và mô tả đúng thiết kế sampling.

Khả năng tái lập. API có thể thay đổi dưới cùng alias và endpoint có thể ngừng phục vụ. Lưu raw, exact prompt, parameters và thời điểm; snapshot/version nếu có. Gói tái lập phải chạy được phần offline không cần API để tái tạo bảng chính từ prediction; phần online ghi rõ điều kiện và chi phí.

18 Sản phẩm cần bàn giao

Gói bàn giao gồm protocol đã khóa; ledger và commit; manifest cohort/fold/mask; model/scaler/vocabulary có checksum; prompt/raw/prediction canonical; evaluator; bảng repair/harm và chi phí; packet chấm mù trước/sau hòa giải; các bảng/hình sinh bằng script; và bản bài nghiên cứu trả lời từng RQ.

Một người khác cần tái tạo được metric, clean sensitivity và đường budget chỉ từ gói prediction, nhãn, mask và script. Giữ lịch sử run và winner lock; không thay raw response hoặc nhãn gốc để làm số liệu khớp. Khi chia sẻ phải kiểm quyền phân phối dataset/model và loại khóa API khỏi log.

Việc đầu tiên thực hiện trong vòng ba ngày là tạo bảng lỗi theo lớp và audit prompt H3-B, cùng cặp P_pre/P_post đúng đối chứng. Sau bước đó mới chốt số truy vấn mới. Hướng nghiên cứu vẫn là một câu hỏi về giá trị bổ sung; kết quả của nó có thể chỉ ra nên dùng LLM có chọn lọc, dùng sau truy vấn, hoặc chưa nên trả thêm chi phí trong setup đang xét.

Bộ mã đi kèm đã chạy đạt 59 unit test cho contract/probe/scoring, canonical export, statistic/routing, matched meta, CLI và request identity trên fixture kỹ thuật. Các kiểm thử không phải benchmark hay bằng chứng về hiệu quả phát hiện lỗ hổng. Chưa chạy GPU/model thật hoặc full pipeline của repository trong môi trường này; cần smoke test development sau tích hợp trước inference xác nhận.

19 Bản đồ cập nhật mã nguồn hiện có

Phạm vi đã kiểm chứng

Đã đọc mã nguồn công khai ở hai snapshot, không thay đổi repository từ xa và chưa chạy pipeline LLM với dữ liệu thật. Snapshot của gói H3-B là commit `c78bacba01b5eb74846c3f79cfdd6d2d878f37dc`; snapshot của pipeline trong thư mục src là `973267ba22a8c8dc8a138345a53b66684af06427`. Các vị trí dưới đây là điểm móc đã xác minh trong mã, không phải cam kết một patch tự động áp dụng được cho mọi bản sao cục bộ.

Các thay đổi ở nhánh H3-B

1. **Giữ nguyên baseline lịch sử và tạo cấu hình nghiên cứu mới.** grace_h3v2/base.py::train_base chọn classifier và ngưỡng trên train nested OOF, ghi winner, candidates, validation_views.npy và train_nested_oof.npy. Đọc đúng base['winner']['name'] trước khi mô tả baseline là LR, MLP hay fusion. Không suy từ tên gate rằng nó chỉ là một hồi quy logistic 768 chiều. Khóa classifier và ngưỡng chung từ manifest cho các policy mới; không thay lock của session cũ.

2. **Thêm P_pre bên cạnh P_post tại hàm fit thật.** grace_h3v2/meta.py::pre_features tạo 34 feature có trước expert. post_features thêm bốn feature từ verdict/validity của expert. fit_meta hiện fit stack_lr trên post feature; chưa có đối chứng cùng LR trên pre feature. Có thể thêm arm mới pre_meta_lr và post_meta_lr trong một module nghiên cứu riêng, dùng cùng StandardScaler + LogisticRegression(C=1, max_iter=2000), cùng nhãn development, cùng fold indices và cùng quy tắc chọn cutoff. Nên tạo các fold bằng common.folds đúng group key rồi dùng chung cho cả hai arm. Các scaler chỉ fit ở phía học của từng fold. Không fit P_pre trên score in-sample của một model đã học chính target đó.

3. **Fallback phải thuộc đúng phép so.** meta.py::output_for hiện fallback về h của classifier đối với stack/router. Giữ cơ chế đó khi tái tạo kết quả lịch sử. Với phép xác nhận P_post–P_pre mới, prediction P_post ở expert-invalid phải bằng prediction P_pre tương ứng; trong OOF dùng prediction P_pre ngoài fold, trong test dùng P_pre đã khóa. Router vẫn fallback về gate chung. Vì vậy không chỉ đổi tên stack_lr thành P_post rồi dùng nguyên logic fallback.

4. **Confidence quanh biên quyết định.** Trong fit_meta và predict_all, confidence hiện là 1 - 2*abs(probability-.5). Với policy mới, tính score từ khoảng cách abs(logit(p)-logit(tau_h)), với clipping p và tau đã khóa. Giá trị nhỏ được ưu tiên gọi expert; không gọi score này là xác suất lỗi. Giữ confidence quanh 0,5 cũ trong bảng lịch sử nếu cần truy vết.

5. **Xuất score đúng nghĩa của từng arm.** grace_h3v2/workflow.py::predict, khoảng dòng 217–223 của snapshot, ghi gate_score=float(probability[i]) vào tất cả file arm. Đây là score của base winner, kể cả dòng thuộc full_grace, stack_lr hay router. Không dùng cột này để tính AUROC cho LLM/stacker. Muốn có score stacker, sửa predict_all trả thêm score và score_semantics của chính arm, rồi chuyển chúng qua workflow.predict. Với expert chỉ có nhãn, ghi score=null. Việc chọn nhãn từ gate score không tạo ra score liên tục của LLM.

6. **Đổi evaluator mà không làm hỏng chọn mô hình.** common.metrics đang gọi grace_h1.metrics.binary_metrics, thư viện nội bộ này trả 0 cho một số trường hợp không xác định. Module mới ghi MCC/precision không xác định là null. Không thay nóng hàm cũ bằng module mới nếu chưa sửa common.choose, vì hàm này đang max(mcc) và so ngưỡng bằng số. Trong protocol mới, loại candidate có metric không xác định khỏi quyền được chọn, ghi lý do; không thay null bằng 0 để tiện xếp hạng. File lịch sử giữ quy ước cũ và được chú thích.

Các thay đổi ở nhánh chuẩn bị prompt và inference

7. **Xuất thành phần trước khi cắt.** grace_h1/pipeline.py::prepare_run đã có target, demo, graph_text và selected ngay trước lời gọi fit_contracted_prompt. Thêm prompt_components.jsonl chứa ID/hash target, target code, graph/evidence nguồn hoặc đường dẫn hash, demo ID/code/nhãn, policy renderer và thứ tự. Không thêm nhãn target. Xuất từ cùng lựa chọn retrieval đã khóa; các probe không truy hồi lại ngoài can thiệp được khai báo. graph_text ở vị trí này đã qua raw_graph_context và có thể đã bị giới hạn prefix; để nghiên cứu full graph/H2 phải lưu graph giàu thuộc tính hoặc graph source hash trước bước này. Không thể khôi phục graph đầy đủ từ một chuỗi prefix đã mất thông tin.

8. **Tách probe khỏi PipelineConfig lịch sử.** PipelineConfig.validate hiện chỉ chấp nhận các arm H1 cũ. Cách ít xâm lấn là một script nghiên cứu đọc prompt_components.jsonl, gọi gracekit.probes, xuất prompt

bundle mới và manifest/hash. Nó không đổi retrieval, encoder hoặc graph cache. Nếu tích hợp probe trực tiếp vào orchestration, thêm registry riêng và config version mới, không lách kiểm tra bằng cách đặt mọi probe tên baseline_whitening.

9. **Đổi contract byte sang token tạo session mới.** grace_h1/pipeline.py và reference.fit_contracted_prompt hiện ghi rõ budget bảo thủ theo UTF-8 byte. Với contract mới, dùng tokenizer và chat template thật để đếm toàn prompt, kể cả instruction và output reserve; lưu byte/ký tự như telemetry phụ. Không ghi nhãn token cho byte budget cũ. Nếu vượt budget, báo lỗi hoặc áp policy cắt đã khóa, không xóa demo tới rỗng một cách im lặng. Hash của contract/model/tokenizer/template phải vào checkpoint và cache key.

10. **Inference chỉ lưu những gì thực sự thu được.** grace_h1/inference.py::run_inference đã xuất record_id, code_sha256, prediction, parse_status, request_status, request_key, prompt_sha256; raw và usage ở file riêng. Đây là nguồn tốt để export. Nếu thêm logprob, lưu nguyên candidate token IDs, token strings, logprob, vị trí assistant và score status; không điền logprob thiếu bằng −30 hoặc score giả. Với API không trả đủ candidate, nhãn có thể hợp lệ trong khi score không có. workflow.checked_expert kiểm request spec và checksum nhưng có nhánh recovery mixed-model lịch sử; lượt xác nhận mới không được đi qua ngoại lệ mixed-model.

Các thay đổi ở pipeline thư mục src

11. **Nối parser mới vào callsite thực tế.** Snapshot có src/evaluator.py::parse_strict_binary ở dòng 144, nhưng LLMEvaluator.evaluate_dataset vẫn gọi parse_llm_prediction khoảng dòng 532. Chỉ thêm hàm strict chưa làm run sử dụng nó. Tạo parser mode/version trong cấu hình mới, truyền finish_reason và chọn parser tại callsite; giữ parser lịch sử để đối soát raw cũ. Tách request exhausted khỏi parse-invalid và khỏi fallback cuối. Không mặc định mọi finish_reason='length' là nhãn hỏng khi contract có max_tokens=1 và đã sinh đủ token 0/1; quy tắc strict cần xét sự hoàn chỉnh của nhãn theo contract.

12. **Thêm hash đầy đủ vào record mới.** evaluate_dataset hiện ghi id, target, prediction, parse_method, is_valid, func_snippet 120 ký tự, raw và timestamp. Snippet không đủ xác nhận danh tính mã đầy đủ. Thêm code_sha256 từ code canonical, prompt_sha256 của prompt gửi thật, run_config_hash, provider/model/version và request status. Khi export kết quả cũ thiếu hash, cần khôi phục identity bằng artifact đáng tin cậy; adapter không tự băm snippet hoặc gán hash canonical để giả vờ đã kiểm.

13. **Bộ dựng prompt cần nhận object không có target label.** Điểm móc là src/prompt_engine.py::build_grace_prompt, build_contrastive_prompt và build_prompt. Chuyển kết quả retrieval thành contract gồm code/graph/demo có nhãn train, sau đó gọi probe bằng object này. Thử giữ code và thay/bỏ nhãn target ở phía dữ liệu: prompt, demo IDs và routing score phải không đổi. Không kiểm bằng cách cấm mọi ký tự “0”/“1” trong prompt vì code và demo hợp lệ vốn có các ký tự đó. Các can thiệp đảo nhãn chỉ sửa trường label do template sinh, không thay chữ vulnerable/safe bên trong mã nguồn hoặc chú thích code.

14. **Giữ kho train khi chạy subset.** src/run_pipeline.py::run_grace_pipeline đã có train_sample_ratio và test_sample_ratio; tuy vậy alias sample_ratio vẫn gán cả hai effective ratio. Lượt mới không truyền alias này; dùng train_sample_ratio=1.0, test_sample_ratio theo target cohort, hoặc danh sách target ID cố định. Ghi hash full retrieval pool vào manifest. Các tỷ lệ chọn mẫu không thay thế một manifest cohort rõ ràng.

Dùng adapter và kiểm tra sau tích hợp

Module gracekit.adapters.export_canonical nhận prediction của một arm và label/code manifest authoritative, yêu cầu cùng tập ID và full code SHA-256. Nó từ chối duplicate ID, hash/nhãn lệch, request lỗi mang prediction và score thiếu ngữ nghĩa. Mặc định score=null; gate_score chỉ được chọn có chủ đích

cho arm gate/base với semantics base_classifier_probability. Group và sampling weight lấy từ canonical manifest. Nhãn thật được ghép ở bước export đánh giá, không quay trở lại prompt/inference.

Các unit test của adapter đã chạy trên fixture kỹ thuật, gồm thiếu/hash sai, duplicate/missing ID, nhãn mâu thuẫn, expert exhausted, final fallback và gate_score bị dùng sai. Các test này xác nhận hành vi exporter, không phải số liệu benchmark. Cần chạy smoke test tích hợp trên 20–100 mẫu development với dữ liệu, tokenizer và server thật trước full inference. Chưa có kiểm chứng end-to-end của repository/model/dataset trong môi trường này.

Nguồn mã đã đối chiếu

https://github.com/kazunguyen/GRACE-H3-BenchmarkEnsembled/tree/c78bacba01b5eb74846c3f79cfdd6d2d878f37dc

Snapshot pipeline src tại commit 973267ba22a8c8dc8a138345a53b66684af06427 được định danh ở [N5].

Phụ lục A Gói mã và cấu hình đi kèm

Gói GRACE_research_kit.zip là phần bổ sung độc lập. Các module chính gồm analysis.py cho metric/paired CI/repair-harm/routing; probes.py cho can thiệp prompt từ selections đã khóa; scoring.py cho candidate thật và full logits; adapters.py cho export có kiểm identity; matched_meta.py cho cặp LR cùng OOF/fallback; identity.py cho request cache/protocol và cli.py cho chạy offline. Script score_local_hf.py là đường scoring tùy chọn cần model local thật.

Các config JSON là template chưa khóa, cố ý còn trường null về model/data/version. Chúng là file thông số đầu vào có thể cập nhật, không phải một experiment lock đã đủ điều kiện xác nhận. Schema canonical phân biệt prediction_status và score_status, nhãn thật chỉ ở file đánh giá. README và INTEGRATION.md mô tả callsite, format và các phần cần nối với code hiện có.

Không sao chép toàn bộ code dài vào Word; mã có thể chạy được được cung cấp trong zip cùng tests và fixture có nhãn fixture_only. Với phiên bản mới, lưu pip freeze sau khi tests/smoke test pass; requirements chỉ đặt mức tương thích tối thiểu, không tự pin toàn bộ GPU stack.

Phụ lục B Các lệnh thực hiện và đầu vào minh họa

Chạy từ thư mục gói mã sau khi giải nén. Các file examples bên dưới là dữ liệu tổng hợp để kiểm luồng, không phải prediction nghiên cứu. CLI không gọi API hoặc tự tìm threshold bằng test.

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python -m gracekit summarize --input examples/gate.jsonl \
  --output demo_output/gate_metrics.json
python -m gracekit repair-harm --gate examples/gate.jsonl \
  --expert examples/expert.jsonl --output demo_output/repair_harm.json
python -m gracekit compare --a examples/gate.jsonl \
  --b examples/expert.jsonl --metric mcc --resampling stratified \
  --replicates 2000 --output demo_output/paired.json
python -m gracekit route --gate examples/gate.jsonl \
  --expert examples/expert.jsonl --q 0.10 \
  --output demo_output/random_route.jsonl
```

Với dữ liệu thật, thay bằng canonical file của cùng cohort, giữ hash/group/weights và chọn resampling theo protocol. compare --a A --b B luôn trả B trừ A. File mẫu protocol.template.json bị validate từ chối cho đến khi điền đủ metadata và đặt locked=true; không bỏ validator để chạy nhanh. Gọi fit_matched bằng Python theo ví dụ README, truyền feature development/frozen gate, không truyền nhãn test.

Đối với inference, adapter cần cung cấp tokenizer/chat-template thật và graph/demo components. Full raw logit scorer không chạy model thay cho code inference; script local HF cần model local đã pin, phần cứng đủ và smoke test riêng. Một pipeline chỉ có nhãn vẫn dùng được analysis/repair/routing, với score=null và không có bảng AUC của expert.

Phụ lục C Mẫu bảng hình và nghiệm thu

Bảng C1 dữ liệu. Mỗi dòng là dataset/cohort; các cột gồm train/dev/test hoặc eligible/sample; N0/N1; số project/commit; số overlap bị loại; tỷ lệ graph parse; tình trạng exposure. Giữ cùng schema cho mọi dataset.

Bảng C2 kết quả chính. Mỗi dòng là arm của cùng cohort; các cột gồm TP/FP/TN/FN, MCC, BA, F1, coverage, expert-query ratio và tiền/1.000 target. Tách unweighted diagnostic và weighted population nếu lấy mẫu phân tầng. Không ghi số chưa chạy thành 0.

Bảng C3 các cặp xác nhận. Ghi contrast, effect size, CI, metric, unit resampling, seed, điều chỉnh nhiều phép kiểm và kết luận theo biên thực tiễn. Cặp P_post–P_pre và benefit–confidence tại 10% đứng đầu, những cặp hậu kiểm được đánh dấu.

Bảng C4 lỗi và chi phí. Ghi R0/H0/R1/H1 tại từng budget; số query theo thuật toán; request/cache/retry; token; training/prepare/inference latency. Có cột N/A và missing được giải thích.

Hình C1 vẽ MCC và BA theo query ratio với gate/full expert làm mốc; thể hiện random routing bằng dải phân bố. **Hình C2** vẽ sửa FN, tạo FN, sửa FP và tạo FP theo budget. **Hình C3** thể hiện paired effect size theo dataset/backbone và CI. Đây là hình cần sinh từ kết quả thực, không dùng số minh họa làm dữ liệu.

Nghiệm thu gồm: đúng ID/hash/nhãn và cohort; không dùng gate_score cho expert; prompt diff đúng khối; không có target label trong object prompt; candidate đủ hoặc score null; current parser thật sự được gọi; threshold/folds lấy từ development; fallback P_post đúng P_pre; repeat bypass cache có replicate identity; metric/CI/coverage có nguồn; 26 nhóm suy luận được xử lý nhất quán; không có tên cá nhân hoặc phân vai trong nội dung đề cương.

Tài liệu nguồn thực nghiệm

[N1] Phản hồi chuyên môn chính thức về nền thực nghiệm GRACE, đối chứng và lộ trình ba đợt, bản cung cấp kèm hồ sơ.

[N2] Báo cáo chung đối soát các phiên bản cài đặt cũ, cập nhật trong các mục Đợt 1 đến Đợt 3; đặc biệt kết quả development và chấm retrieval.

[N3] Báo cáo H1, H2, H3-A và H3-B; snapshot/đối soát cập nhật 24/10/2026. Số liệu lịch sử dùng các mục 4.6, 6.1.2, 6.2 và 6.2.2. Số liệu phải tái tạo từ artifact trước công bố.

[N4] Đề cương bổ sung về tái lập có chẩn đoán trên LLM open weight, ngày 28/10/2026, bản cung cấp kèm hồ sơ. Các phép thử và khả năng suy luận được đánh giá theo phạm vi bằng chứng, không coi giả thuyết là kết quả.

[N5] Snapshot source H3-B c78bacba01b5eb74846c3f79cfdd6d2d878f37dc và pipeline src 973267ba22a8c8dc8a138345a53b66684af06427, đối chiếu cho mục 19. Bản đang dùng cục bộ có thể khác; cần diff trước khi nối hook.

Tài liệu tham khảo

[T1] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai. GRACE Empowering LLM-based software vulnerability detection with graph structure and in-context learning. Journal of Systems and Software 212, 112031, 2024. DOI 10.1016/j.jss.2024.112031. https://doi.org/10.1016/j.jss.2024.112031 ; mã tác giả https://github.com/P-E-Vul/GRACE

[T2] Y. Yang và cộng sự. DLAP A Deep Learning Augmented Large Language Model Prompting framework for software vulnerability detection. Journal of Systems and Software 219, 112234, 2025. DOI 10.1016/j.jss.2024.112234. https://arxiv.org/abs/2405.01202

[T3] Z. Liu, J. Chen, Z. Huang. Software vulnerability detection via multimodal retrieval and hierarchical decision making. Science of Computer Programming, 103491, 2026. DOI 10.1016/j.scico.2026.103491. https://www.sciencedirect.com/science/article/abs/pii/S0167642326000572

[T4] H. Mozannar, D. Sontag. Consistent Estimators for Learning to Defer to an Expert. ICML, PMLR 119, 7076–7087, 2020. https://proceedings.mlr.press/v119/mozannar20b.html

[T5] H. Narasimhan, W. Jitkrittum, A. K. Menon, A. Rawat, S. Kumar. Post-hoc estimators for learning to defer to an expert. NeurIPS 35, 2022. https://papers.neurips.cc/paper_files/paper/2022/hash/bc8f76d9caadd48f77025b1c889d2e2d-Abstract-Conference.html

[T6] Z. Sun và cộng sự. Ensembling Large Language Models for Code Vulnerability Detection An Empirical Evaluation. arXiv 2509.12629v2, 2025. Preprint. https://arxiv.org/abs/2509.12629

[T7] F. C. Grasso, P. Liguori, R. Natella, D. Cotroneo. Enhancing LLM-Based Vulnerability Detection via Representation-Aware Logical Ensemble. arXiv 2610.33756, 24/10/2026; trang tác giả ghi accepted QRS 2026. https://arxiv.org/abs/2610.33756

[T8] Y. Ding và cộng sự. Vulnerability Detection with Code Language Models How Far Are We. ICSE 2025. DOI 10.1109/ICSE55347.2025.00038. https://arxiv.org/abs/2403.18624 ; dữ liệu và code https://github.com/DLVulDet/PrimeVul

[T9] Y. Li, W. Qi, X. Wang, F. Yu, X. Wang. Revisiting Pre-trained Language Models for Vulnerability Detection. AsiaCCS 2026, 1540–1554. DOI 10.1145/3779208.3785291. https://arxiv.org/abs/2507.16887

[T10] F. Weissberg, L. Pirch, E. Imgrund, J. Möller, T. Eisenhofer, K. Rieck. LLM-based Vulnerability Discovery through the Lens of Code Metrics. ICSE 2026. DOI 10.1145/3744916.3764574. https://www.mlsec.tu-berlin.de/docs/2026-icse.pdf

[T11] S. Kaniewski, T. Krämer, J. Bächle, M. Enzweiler, M. Menth, T. Heer. Retrieve Reproduce Reveal Dissecting Retrieval-Augmented Software Vulnerability Detection. arXiv 2609.37669, 2026; replication package chính thức. https://github.com/hs-esslingen-it-security/RAG4SVD

[T12] M. Cichoń, B. Dmitruk. A Function-Level Vulnerability Score Measures Flag Rate More Than the Model Protocol Effects on Paired Benchmarks. arXiv 2609.32890v2, 19/10/2026; trang tác giả ghi NeurIPS 2026 Workshop TAE. https://arxiv.org/abs/2609.32890

[T13] scikit-learn. StackingClassifier và hướng dẫn cross-validation. Tài liệu chính thức. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html ; OOF là nguyên tắc dùng trong thiết kế meta-model.

[T14] statsmodels. mcnemar. Tài liệu chính thức về kiểm định bảng ghép cặp. https://www.statsmodels.org/stable/generated/statsmodels.stats.contingency_tables.mcnemar.html

[T15] ESEM. Technical Track và các hình thức nghiên cứu thực nghiệm. Trang hội thảo, dùng để xác định phạm vi chuyên môn, không phải thông báo kỳ đang nhận bài. https://conf.researchr.org/track/eseiw-2026/eseiw-2026-esem---technical-track

[T16] SANER. Research Track và các track của SANER 2027. Trang hội thảo, dùng để xác định phạm vi chuyên môn. https://conf.researchr.org/track/saner-2027/saner-2027-papers ; https://conf.researchr.org/home/saner-2027

[T17] vLLM. Reproducibility và Batch Invariance. Tài liệu chính thức, kiểm tra theo version sử dụng. https://docs.vllm.ai/en/stable/usage/reproducibility/ ; https://docs.vllm.ai/en/stable/features/batch_invariance/

[T18] vLLM. Structured Outputs. Tài liệu chính thức về constrained decoding và thay đổi API. https://docs.vllm.ai/en/stable/features/structured_outputs/

[T19] Qwen. Qwen2.5-Coder-32B-Instruct-AWQ model card. https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct-AWQ

[T20] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, L. Zettlemoyer. Rethinking the Role of Demonstrations What Makes In-Context Learning Work. EMNLP 2022. https://aclanthology.org/2022.emnlp-main.759/

[T21] J. Wei, J. Wei, Y. Tay và cộng sự. Larger language models do in-context learning differently. arXiv 2303.03846, 2023. https://arxiv.org/abs/2303.03846

[T22] vLLM. Chat completion protocol và candidate token logprob interface. Source API theo phiên bản. https://github.com/vllm-project/vllm/blob/main/vllm/entrypoints/openai/chat_completion/protocol.py ; phải pin revision trước inference.

[T23] Qwen. Qwen2.5 Speed Benchmark. Tài liệu tác giả, có phần cứng, version, input/output length và batch size. https://qwen.readthedocs.io/en/v2.5/benchmark/speed_benchmark.html

Các nguồn trực tuyến được đối chiếu đến ngày 28/10/2026. Preprint, thông tin accepted từ trang tác giả và ấn phẩm proceedings có DOI được phân biệt. Tính mới, phiên bản phần mềm, CFP và chỉ mục/quartile cần được kiểm tra lại đúng thời điểm nộp hoặc chạy.